

More Models & Unsupervised Learning

Day 3: Detailed learning notes

Testing models fairly, finding patterns in unlabelled data, and learning from a small number of answers.

A companion to all three Day 3 notebooks

Model showdown; unsupervised learning; and semi-supervised learning.

How to study with this document

Keep the relevant notebook open. Read the explanation, run the code, and take a moment to describe the output in your own words. The worked examples explain the mathematics, Python syntax, model settings, and meaning of the results. Prepared from the supplied 35-slide deck, three

notebooks, and Day 3 speaker notes. Core notebook results and exercises were checked by running the code. Differences from the supplied results are identified where they occur.

Contents

1	Starting Day 3	4
1.1	What connects today's topics?	4
1.2	The essential words from Days 1 and 2	4
1.3	The notebooks and lesson sequence	4
1.4	Running the code and reading these notes	5
2	Notebook 1: getting to know the classification data	5
2.1	Imports and setup	5
2.2	What does one row represent?	6
2.3	Reading the first scatter plot	6
2.4	Creating the initial split	7
3	Understanding overfitting and fair testing	8
3.1	Underfitting, overfitting, and the training gap	8
3.2	Watching a decision tree grow	8
3.3	Why changing the split changes the score	10
3.4	Five-fold cross-validation, step by step	10
3.5	What does 97.8% plus or minus 1.0% mean?	11
4	How the six classifiers work	12
4.1	Logistic regression: a weighted classification rule	12
4.2	K-nearest neighbours: consulting nearby examples	12
4.3	Decision trees: sequences of feature thresholds	12
4.4	Ensembles: combining several fitted models	13
4.5	Random forest: many varied decision trees	13
4.6	Gradient boosting: adding corrections	13
4.7	Support vector machines: boundaries and margins	14
5	Comparing models and interpreting their mistakes	15
5.1	The complete comparison loop	15
5.2	The box plot of fold scores	16
5.3	Reading the forest's importance plot	16
5.4	The confusion matrix and class-specific recall	17
6	Tuning settings with GridSearchCV	19
6.1	Parameters and hyperparameters	19
6.2	Reading the search table	19
6.3	A fresh experiment with a genuinely reserved test set	20
7	Titanic: putting preparation inside the model pipeline	22
7.1	Why preprocessing belongs inside cross-validation	22
7.2	Building the two preprocessing branches	22
7.3	What the cross-validation loop actually fits	23
7.4	Predicting from raw passenger rows	24
8	Notebook 1 exercises, with explanations	25
8.1	Exercise 1: tune a random forest	25
8.2	Exercise 2: choose the KNN neighbourhood	25
8.3	Exercise 3: optional XGBoost comparison	26
9	Notebook 2: finding customer groups with K-Means	26

9.1	What can we learn without target labels?	26
9.2	Loading the customer table	27
9.3	Distances and centroids	28
9.4	The K-Means learning loop	28
9.5	Why feature scales matter here	29
9.6	Describing the groups in ordinary language	30
9.7	Choosing k: elbow and silhouette	31
10	Hierarchical clustering and the dendrogram	32
10.1	Building groups through successive merges	32
10.2	Comparing two cluster assignments	33
11	PCA: representing many measurements with fewer numbers	34
11.1	Turning an image into a feature vector	34
11.2	What a principal component means	34
11.3	The two-component digit plot	35
11.4	Choosing a number of components	36
11.5	Scaling changes the PCA problem	37
11.6	How dimensionality affects classification	38
11.7	When the PCA representation helps	38
12	Detecting unusual motor readings	39
12.1	Building the simulated example	39
12.2	How Isolation Forest identifies unusual points	40
12.3	Understanding the detection result	41
13	Notebook 2 exercises, with explanations	42
13.1	Exercise 1: include age in customer clustering	42
13.2	Exercise 2: change the alert threshold	43
13.3	Exercise 3: breast cancer measurements in two dimensions	43
14	Notebook 3: learning when only a few labels are available	44
14.1	Features can be available before their answers	44
14.2	Start by separating the test images	45
14.3	Hiding all but 50 labels	45
14.4	The baseline using the 50 confirmed answers	46
15	Self-training: learning from selected predictions	46
15.1	Following one self-training round	46
15.2	The code and learned attributes	47
15.3	Why confidence needs interpretation	47
16	Label spreading: using the geometry of all training images	48
16.1	A graph of similar examples	48
16.2	Fitting and evaluating label spreading	48
16.3	The supplied results and the verification run	48
16.4	Inspecting individual inferred labels	49
16.5	Comparing the four approaches	50
17	How many labels do we need?	51
17.1	The label-budget experiment	51
17.2	Exercise 1: change the graph neighbourhood	52
17.3	Exercise 2: change the self-training threshold	53
17.4	Exercise 3: why messy data can make spreading fail	53

18 Bringing the lesson together	54
18.1 Choosing an initial method	54
18.2 Answers to the classroom discussion questions	54
18.3 Before moving to Day 4	55
19 A short self-check, with worked answers	55
19.1 Questions	55
19.2 Worked answers	56
20 Practical troubleshooting	56
21 Glossary for quick revision	57
22 Coverage, verification, and further reading	59
22.1 Where the supplied material appears	59
22.2 What was checked	59
22.3 Sources and documentation	59

1 Starting Day 3

Material covered: Slides 1–3; introductions and running order in the speaker notes.

1.1 What connects today's topics?

Suppose you have trained a model and obtained 95% accuracy. Before deciding that it works well, you need to know which examples produced that score, how those examples were chosen, and whether information about their answers influenced training. You also need something to compare the score with. These questions lead into the first half of Day 3.

The second half begins with a different situation: you have measurements, images, or customer records, but few or none of their answers. You will explore how the arrangement of those examples can still be useful. Customers may form groups, images may have repeated patterns, and unusual sensor readings may sit far from the main collection.

Today's tasks use four datasets. The breast cancer measurements support a classification comparison. The Titanic data revisits preprocessing and prediction. Mall customer records introduce clustering. Handwritten digit images support dimensionality reduction and semi-supervised learning. A small simulated motor dataset introduces anomaly detection.

1.2 The essential words from Days 1 and 2

A **sample** is one example, represented by one row of a table. A **feature** is an input measurement, represented by a column. A **label** or **target** is the answer supplied during supervised learning. We usually call the feature table X and the target array y .

Training means adjusting a model using the available training examples. **Prediction** means applying the fitted model to an example. **Evaluation** means comparing predictions with known answers using a chosen measure. **Generalisation** describes how well the learned relationship works on new examples from the intended setting.

For classification, accuracy is

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of evaluated examples}}.$$

If 90 of 100 predictions are correct, accuracy is $90/100 = 0.90 = 90\%$. A **baseline** provides an initial reference score. On Day 2, predicting the majority Titanic class gave approximately 61.5%. Every model should be assessed against a relevant starting point.

1.3 The notebooks and lesson sequence

Notebook	Contents	When to use it
1: Model show-down	Overfitting, cross-validation, six classifiers, tuning, Titanic pipelines	Parts 1–2 at slide 9; Parts 3–5 at slide 18.
2: Unsupervised	Customer clustering, hierarchical clustering, PCA, motor anomalies	All parts at slide 27.
3: Semi-supervised	Hidden labels, baseline, self-training, label spreading, label budgets	All parts at slide 32.

The three-hour session gives roughly 10 minutes to the recap, 35 to fair testing, 40 to stronger models, a 10-minute break, 50 to unsupervised learning, 25 to semi-supervised learning, and 10 to the closing discussion. These notes provide additional study time around each practical block.

1.4 Running the code and reading these notes

Run notebook cells from top to bottom. A cell can depend on a variable created much earlier. For example, Notebook 1 reuses `cv` after the first hands-on break. Opening another notebook does not transfer that variable into its runtime.

The breast cancer and digit datasets are included with scikit-learn. The mall CSV and Seaborn's Titanic loader may need an internet connection. The optional XGBoost exercise needs the separate XGBoost package. A CPU is sufficient for today's examples.

A code cell creates **objects** and carries out operations. An **object** holds data or behaviour, such as a fitted model. A **method** is an operation attached to an object, such as `model.fit`. Text after `##` is a comment. Indentation groups the instructions inside a loop or function. A name followed by parentheses calls a function or constructs an object.

The code listings follow the supplied notebooks. Where their teaching shortcuts need qualification, the explanation states it beside the example. Output percentages describe these particular experiments. The core runs here used scikit-learn 1.8.0, NumPy 2.3.5, Pandas 2.2.3, and Seaborn 0.13.2. The optional XGBoost import followed the notebook's missing-package branch.

2 Notebook 1: getting to know the classification data

Material covered: Notebook 1, Part 1; slides 6, 12 and 15.

2.1 Imports and setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold,
    GridSearchCV
from sklearn.pipeline import make_pipeline, Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
print("Ready!")
```

`import numpy as np` makes NumPy available through the short name `np`. NumPy handles numerical arrays. Pandas, imported as `pd`, handles tables. Matplotlib provides plotting functions through `plt`; Seaborn, called `sns`, adds convenient statistical plots. The plotting theme gives charts a white background and grid lines.

The imports from scikit-learn gather the tools needed throughout the notebook. Dataset loaders provide examples. Model-selection tools create splits, run cross-validation, and search settings. Pipelines combine processing steps. The scaler changes feature units. The remaining imports provide the classifiers.

`warnings.filterwarnings("ignore")` hides warning messages. This keeps the classroom output short, but it can also hide useful information about failed convergence or changing software behaviour. When investigating a surprising result, temporarily remove that line and read the warning. `print("Ready!")` simply confirms that execution reached the end of the setup cell.

2.2 What does one row represent?

```
bc = load_breast_cancer()
X, y = bc.data, bc.target

df = pd.DataFrame(X, columns=bc.feature_names)
df["diagnosis"] = pd.Series(y).map({0: "malignant", 1: "benign"})

print("Shape:", X.shape)
print(df["diagnosis"].value_counts())
df.head()
```

`load_breast_cancer()` returns a collection containing the numerical measurements, target codes, names, and a dataset description. `bc.data` becomes `X` and `bc.target` becomes `y`. The statement `X, y = ...` assigns two values at once.

The feature array has shape (569, 30). There are 569 examples and 30 measurements per example. The target has one entry for each row. Its codes are 0 for malignant and 1 for benign. These codes identify the categories; their numerical order has no clinical meaning.

The measurements describe cell nuclei in digitised images of fine-needle aspirates. Ten basic measurements, including radius, texture, perimeter, area, and smoothness, are summarised three ways: the mean, the standard error, and a “worst” value. Here the worst value is the mean of the three largest measured values. This produces $10 \times 3 = 30$ features. The dataset documentation in Reference 1 explains their definitions.

`pd.DataFrame` gives the array named columns. The diagnosis column is added for inspection: `map` replaces each numerical target with a readable category. It is kept outside `X`, which already contains only the input measurements. Including diagnosis among the inputs would expose the answer.

`value_counts()` reports 357 benign examples and 212 malignant examples. `head()` displays the first five rows. The displayed table has 31 columns because it contains the 30 measurements plus the diagnosis column. The model’s input still has 30.

Always predicting benign would achieve about $357/569 = 62.7\%$ on the full dataset. This is a useful first reference when interpreting the much higher classifier scores later.

2.3 Reading the first scatter plot

```
sns.scatterplot(data=df, x="mean radius", y="mean concave points", hue="diagnosis", alpha=0.7)
plt.title("Two of the 30 measurements")
plt.show()
```

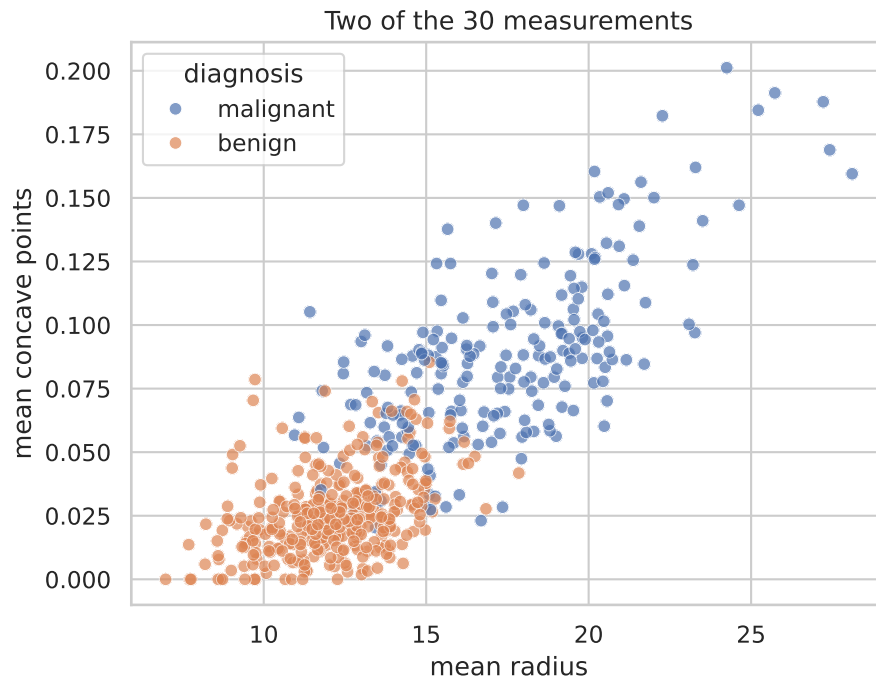


Figure 1: Two measurements from the breast cancer data. Each dot is one example.

The horizontal position gives mean radius. The vertical position gives mean concave points, a measurement related to indentations along the nucleus boundary. `hue="diagnosis"` assigns different colours to the two categories. `alpha=0.7` makes points partly transparent, which helps when several overlap.

The groups occupy noticeably different regions, with some overlap. This suggests that useful predictive structure exists. A plot of two columns gives only a partial view of the 30-dimensional input. It cannot establish that every example can be separated by one straight boundary.

2.4 Creating the initial split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print("Train:", X_train.shape, "| Final test:", X_test.shape)
```

`test_size=0.2` reserves approximately one fifth of the examples. The result is 455 training rows and 114 test rows, each with 30 columns. `stratify=y` keeps the class proportions close to their original proportions in both subsets. The test subset contains 42 malignant and 72 benign examples.

`random_state=42` controls the pseudo-random splitting process so that rerunning this experiment produces the same split in a compatible environment. The number 42 has no special predictive power. Reproducibility helps us explain a result; it does not make one split representative of every future situation.

The four returned objects remain aligned. The first training row has its answer at the first position of `y_train`, and the same relationship holds for the test arrays.

A detail to keep in mind. The notebook calls this a final test set, then uses it several times for demonstrations and model comparisons. Once those scores guide decisions, it has a validation role in practice. Later sections show how to keep a separate final evaluation untouched in a fresh experiment.

3 Understanding overfitting and fair testing

Material covered: Slides 4–9; Notebook 1, Part 2; speaker discussion of bias and variance.

3.1 Underfitting, overfitting, and the training gap

A model **underfits** when its learned relationship is too limited to capture useful structure. For example, a tree allowed to ask only one question may struggle with a task that depends on several interacting measurements. Training and validation performance can both be poor. Inadequate features, strong regularisation, and incomplete optimisation can also produce this pattern.

A model **overfits** when it adapts too closely to details specific to its training examples. Some of those details reflect noise or coincidences. Training performance can then be much better than performance on new examples.

The exam analogy helps: practice questions contain examples of a topic. Successful preparation develops a method that works on unfamiliar questions. Repeated exposure to the exact practice answers can produce a high practice score while leaving that broader skill weak.

A model with 99% training accuracy and 70% test accuracy has a 29-percentage-point gap. Overfitting is one plausible explanation. A changed data distribution, inconsistent preprocessing, or a problem with the split can also create a large gap. The scores tell us where to investigate.

Bias describes systematic error associated with a model’s assumptions. **Variance** describes how much its fitted predictions change when the training sample changes. A very restrictive model can miss patterns consistently. A highly flexible model can respond strongly to small changes in the sample. The bias–variance trade-off is a way to reason about these tendencies. It does not mean that every accuracy curve follows a perfectly smooth shape.

3.2 Watching a decision tree grow

A decision tree asks a sequence of questions such as “is mean radius at most this threshold?” Each question divides examples into branches. A **leaf** contains the final prediction. Depth counts the number of decisions along the longest path from the root to a leaf.

A depth-one tree has one split. A depth-two tree can ask a further question on each branch. As depth grows, the tree can describe smaller regions and more specific combinations of measurements. `max_depth=None` removes this particular depth limit; other stopping rules still apply.

```
depths = [1, 2, 3, 4, 5, 6, 8, 10, None]
rows = []
for d in depths:
    tree = DecisionTreeClassifier(max_depth=d, random_state=42).fit(X_train, y_train)
    rows.append({"max_depth": "None" if d is None else d,
                 "train": tree.score(X_train, y_train),
                 "test": tree.score(X_test, y_test)})
curve = pd.DataFrame(rows)
curve.round(3)
```

`depths` stores the settings to try. The empty list `rows` collects the result of each experiment. The `for` loop creates a fresh tree for each depth, fits it on the same training data, then evaluates it on both subsets. For these classifiers, `score(X, y)` returns accuracy.

Each dictionary contains a depth label and two scores. `append` adds the dictionary to the list. Pandas turns the list into a table. `round(3)` changes the displayed precision to three decimal places;

it does not retrain the model.

Maximum depth	Training accuracy	Test accuracy
1	92.3%	92.1%
2	95.8%	89.5%
3	97.6%	93.9%
4	98.7%	93.9%
5	99.3%	92.1%
6	99.8%	91.2%
8, 10, or None	100.0%	91.2%

```
plt.plot(curve["max_depth"].astype(str), curve["train"], marker="o", label="training")
plt.plot(curve["max_depth"].astype(str), curve["test"], marker="o", label="test")
plt.xlabel("max depth of the tree")
plt.ylabel("accuracy")
plt.title("Deeper trees memorise the training data")
plt.legend()
plt.show()
```

`astype(str)` converts the mixed depth labels to strings so they can share one categorical axis. Two calls to `plot` put the training and test curves on the same chart. Markers identify measured values; the connecting lines help the eye follow the sequence.

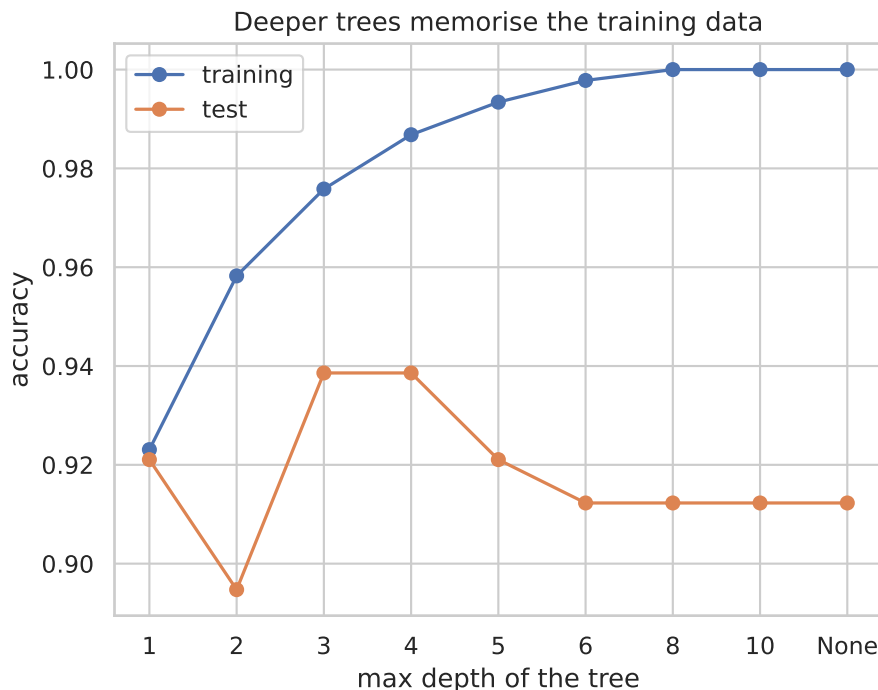


Figure 2: Increasing tree depth improves the training fit while test accuracy levels off and falls.

The best displayed test score occurs at depths three and four. The unrestricted tree fits every training label and makes more errors on the held-out rows. This illustrates the generalisation gap. Notice also that depth two performs worse on this test split than depth one: empirical scores can move irregularly.

The slide version rounds scores to whole percentages and starts its bars at 85%. Check the axis

before judging the size of a visual difference. A truncated axis makes small score differences look larger.

Pause and try. Would you select depth three from this chart and then describe its 93.9% as a completely untouched final evaluation?

Selecting the depth uses information from the chart. The score has therefore contributed to model development. Use validation data or cross-validation to choose depth, then evaluate the selected procedure on separate final data.

3.3 Why changing the split changes the score

```
scores = []
for seed in range(10):
    Xa, Xb, ya, yb = train_test_split(X, y, test_size=0.2, random_state=seed, stratify=y)
    tree = DecisionTreeClassifier(random_state=42).fit(Xa, ya)
    scores.append(tree.score(Xb, yb))
    print(f"split {seed}: {scores[-1]:.1%}")

print(f"\nLowest {min(scores):.1%}, highest {max(scores):.1%}")
```

`range(10)` produces integers zero through nine. Each integer becomes the random seed for a new split of the full dataset. The tree's own seed stays fixed at 42. This isolates the effect of changing which examples enter training and testing as far as this experiment allows.

The names `Xa`, `Xb`, `ya`, and `yb` hold the current split. Each score is added to `scores`. The index `[-1]` retrieves the most recently added entry. The formatting `.1%` displays a proportion as a percentage with one decimal place.

The ten accuracies are 92.1%, 95.6%, 95.6%, 93.9%, 87.7%, 93.0%, 96.5%, 93.0%, 87.7%, and 93.9%. The difference between the minimum and maximum is 8.8 percentage points. Some test subsets contain examples the model finds easier; the training subsets also contain different information.

Every split contains 114 test examples, so one additional correct prediction changes accuracy by approximately $100/114 = 0.88$ percentage points. Small changes in the number of errors can visibly move the score.

Trying seeds until a model looks good biases the reported result. The useful lesson from this demonstration is the variability of the estimate. For the model comparison that follows, every candidate uses the same fold assignments.

3.4 Five-fold cross-validation, step by step

Cross-validation repeatedly trains and evaluates a model within the development data. With five folds, divide the 455 training examples into five groups of 91. Then carry out five runs:

Run	Training folds	Validation fold
1	B, C, D, E	A
2	A, C, D, E	B
3	A, B, D, E	C
4	A, B, C, E	D
5	A, B, C, D	E

Each run fits a fresh model on 364 examples and checks it on 91. Every development example is

evaluated once by a model that did not train on that example. Each example also appears in the training data of four other runs. The final test data is outside this process.

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

logreg = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
fold_scores = cross_val_score(logreg, X_train, y_train, cv=cv)

print("Each fold:", np.round(fold_scores, 3))
print(f"Average: {fold_scores.mean():.1%} +/- {fold_scores.std():.1%}")
```

`StratifiedKFold` preserves approximate class proportions in each fold. `n_splits=5` requests five folds. `shuffle=True` mixes row order before assigning the folds; `random_state` makes that mixing reproducible.

`make_pipeline` puts a scaler before logistic regression. Inside each cross-validation run, the scaler learns its mean and standard deviation from that run's training rows. The validation rows are transformed using those learned values. This avoids sharing validation statistics with the fitting process.

`cross_val_score` fits separate copies of this pipeline and returns five scores. With these classifiers and no explicit `scoring` argument, the score is accuracy. It does not leave the original `logreg` object fitted on the whole dataset.

3.5 What does 97.8% plus or minus 1.0% mean?

The fold accuracies are approximately 96.7%, 98.9%, 97.8%, 98.9%, and 96.7%. Their average is

$$\bar{s} = \frac{s_1 + s_2 + s_3 + s_4 + s_5}{5} \approx 0.978.$$

Here s_i means the score from fold i . The symbol $\bar{s}$ means the average score. The code calculates the spread using NumPy's default standard deviation:

$$\text{spread} = \sqrt{\frac{1}{5} \sum_{i=1}^5 (s_i - \bar{s})^2} \approx 0.010.$$

For each score, subtract the average, square the difference, average those squared differences, and take the square root. A larger value means the observed fold scores differ more from one another.

The printed $97.8\% \pm 1.0\%$ reports a mean and a fold standard deviation. It is not a confidence interval or a promise about the next dataset. The five models have overlapping training sets, so their results are related. With only five scores, the spread is also a limited description of variability.

Five-fold CV gives a broader view than one particular held-out subset. More folds require more fitting and change the sizes of the training and validation subsets. Increasing the number of folds does not guarantee a more stable estimate in every setting. Five and ten are common starting choices for datasets of independent examples.

If rows come from the same person, machine, or household, group-aware splitting may be needed. If the task predicts future events, splitting by time may be needed. Stratification balances class proportions; it does not handle those dependencies by itself.

4 How the six classifiers work

Material covered: Slides 10–14; Notebook 1, Part 3.

4.1 Logistic regression: a weighted classification rule

Logistic regression uses a weighted combination of the input features. For two classes, its basic calculation can be written as

$$z = b + w_1x_1 + w_2x_2 + \cdots + w_px_p, \quad p(y = 1 \mid x) = \frac{1}{1 + e^{-z}}.$$

The x values are the measurements of one example. The w values are learned weights; b is an intercept that shifts the rule. There are p input features. The exponential function e^{-z} converts the score through a smooth curve into a value between zero and one.

With a probability threshold of 0.5, a positive score selects class one. In the breast cancer data, class one means benign. The boundary corresponds to $z = 0$, which is a straight line in two dimensions and a hyperplane in higher dimensions.

Scaling helps because the input measurements use different numerical ranges and the model applies regularisation to its coefficients. Logistic regression itself uses a weighted linear score. It does not find the nearest training examples. `max_iter=5000` permits the numerical optimiser up to 5,000 iterations; training can stop earlier when the stopping criterion is met.

A coefficient describes an association conditional on the other included features. With standardised inputs, it relates to a change of one training standard deviation in that feature. Correlated measurements can complicate interpretation, and a coefficient alone does not establish a causal effect.

4.2 K-nearest neighbours: consulting nearby examples

KNN stores the training examples and predicts from the labels of nearby examples. With $k = 5$, imagine the five nearest points contain four benign labels and one malignant label. The default uniform voting rule predicts benign.

Closeness depends on the feature representation. For two features, Euclidean distance is

$$d(a, b) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2}.$$

Subtract matching measurements, square the differences, add them, and take the square root. The same calculation extends to 30 measurements. A feature whose numerical values vary widely can dominate this distance, so the notebook scales the inputs.

A small k gives individual training examples more influence. A larger k averages over a wider neighbourhood and can smooth away useful local structure. Cross-validation helps assess the choice.

4.3 Decision trees: sequences of feature thresholds

The decision tree from the depth experiment is also included in the showdown. At each node, it searches for a feature and threshold that create useful child groups. Classification trees commonly measure how mixed the labels are in a node using Gini impurity:

$$G = 1 - \sum_c p_c^2.$$

Here p_c is the fraction of examples in class c . A node containing only benign examples has impurity zero. With equal numbers of two classes, $G = 1 - (0.5^2 + 0.5^2) = 0.5$. A split is useful when its child nodes have lower weighted impurity.

Trees can represent interactions: one question can change which question matters next. They are largely unaffected by an ordinary positive rescaling of a feature because its ordering remains the same. An unrestricted tree can also adapt to very small groups of examples, which explains its sensitivity to the training sample.

4.4 Ensembles: combining several fitted models

An ensemble combines several models into one prediction procedure. The jar-of-sweets activity illustrates why this can help. Different people may make errors in different directions, so their average can be closer to the actual count. If everyone makes the same error, averaging preserves that error. Useful diversity matters.

Bagging fits models on resampled versions of the training data. Each model can be fitted independently. The combined prediction reduces some of the instability of an individual model. **Boosting** builds a sequence: each new model is fitted using information about the current ensemble's remaining error.

4.5 Random forest: many varied decision trees

The forest in the notebook contains 200 trees. For each tree, the usual bootstrap procedure samples training rows with replacement. A row may appear more than once in that sample, and some original rows may be absent. At each split, the tree searches among a random subset of candidate features. These two sources of variation help the trees make different errors.

The slide's five-tree example has three malignant votes and two benign votes, so the illustrated majority is malignant. In scikit-learn's implementation, the forest averages the trees' class-probability estimates and selects the class with the largest average. Pure leaves give the simple voting example directly; mixed leaves allow softer contributions.

`n_estimators=200` controls the number of trees. `max_depth` controls each tree's maximum depth when specified. A larger forest can make averaging more stable, but it requires more computation and storage. It can still overfit or fail on a changed population.

Forest **feature importance** measures how much features contributed to impurity reductions across its trees. These values help describe what the fitted forest used. Correlated features can share or substitute for that contribution, and features with many possible split values can receive inflated importance. Read the plot as a model summary, with those limitations in mind.

4.6 Gradient boosting: adding corrections

Imagine a predictor currently outputs 70 for an example whose true numerical target is 100. The residual, actual value minus prediction, is 30. With squared-error regression, a new tree can be trained to predict residuals, and a fraction of its output can be added to the previous prediction.

A simplified update is

$$F_m(x) = F_{m-1}(x) + \eta h_m(x).$$

F_{m-1} is the ensemble before the new step, h_m is the new tree, and η is the learning rate. If the new tree predicts a correction of 20 and $\eta = 0.1$, the prediction moves from 70 to 72. Repeated small steps can build a flexible function.

The classification example uses a classification loss. Its trees are fitted to quantities related to the negative gradient of that loss. The gradient indicates how to change the current scores to reduce error. The residual example gives the intuition without requiring the classification derivation.

Small trees, the number of trees, and the learning rate work together. Too many corrections or excessively flexible trees can fit noise. XGBoost, LightGBM, and CatBoost are separate libraries with different implementations and features in the broader boosting family. Their names in the slides introduce options for later study. This notebook's main comparison uses scikit-learn's `GradientBoostingClassifier`.

4.7 Support vector machines: boundaries and margins

Picture two groups of points with a dividing line between them. The **margin** describes the gap between the decision boundary and nearby points that constrain it. A support vector machine balances a wide margin with penalties for training points that violate that margin.

The influential training points are called **support vectors**. In a simple perfectly separable picture, they lie on the margin boundaries. In a soft-margin fit, points inside the margin or on the wrong side can also be support vectors. A wider margin is a useful learning principle, but its success on new data must still be measured.

An RBF kernel allows a nonlinear decision boundary. Its similarity calculation is

$$K(x, x') = \exp(-\gamma \|x - x'\|^2).$$

x and x' are two feature vectors. The squared distance between them is multiplied by γ , negated, and passed to the exponential function. Close examples have similarity nearer one. Distant examples have similarity nearer zero.

The kernel lets the model compute relationships associated with a richer feature space without explicitly constructing all its coordinates. This is the kernel trick. Scaling affects the distances and therefore matters for this model.

C controls the penalty for margin violations. A larger C puts more pressure on the fit to satisfy the training examples. A smaller C permits more violations in exchange for stronger regularisation. γ controls how locally each point influences RBF similarity. A large value can create very local, intricate boundaries. A small value creates broader influence. Their effects interact, so the notebook tunes them together.

5 Comparing models and interpreting their mistakes

Material covered: Slide 15; Notebook 1, Part 3.

5.1 The complete comparison loop

```
models = {
    "Logistic regression": make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000)),
    "KNN (k=5)":          make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5)),
    "Decision tree":      DecisionTreeClassifier(random_state=42),
    "Random forest":      RandomForestClassifier(n_estimators=200, random_state=42),
    "Gradient boosting":  GradientBoostingClassifier(random_state=42),
    "SVM (RBF)":          make_pipeline(StandardScaler(), SVC()),
}

results, all_folds = [], {}
for name, model in models.items():
    s = cross_val_score(model, X_train, y_train, cv=cv)
    all_folds[name] = s
    model.fit(X_train, y_train)
    results.append({"model": name, "CV mean": s.mean(), "CV spread": s.std(),
                  "final test": model.score(X_test, y_test)})
    print(f"{name:20s} CV {s.mean():.1%} +/- {s.std():.1%}")

showdown = pd.DataFrame(results).set_index("model").sort_values("CV mean", ascending=False)
showdown.round(3)
```

The dictionary `models` connects readable names to unfitted estimators. Models that benefit from standardisation receive a scaler in a pipeline. Tree-based models operate directly on the numerical measurements.

`models.items()` yields each name and estimator. The loop passes every candidate through the same `cv` object, so its five validation subsets are the same. `all_folds` stores the individual fold scores for plotting. `results` stores summary rows.

After CV, `model.fit` fits that named estimator on all 455 development examples. This is a separate fit from the five CV fits. The notebook then also calculates its score on the 114 held-out examples. The table is indexed by model name and sorted by descending CV mean.

Model	CV mean	Fold spread	Held-out score
Logistic regression	97.8%	1.0 pp	98.2%
SVM (RBF)	96.9%	1.5 pp	98.2%
Random forest	96.3%	1.8 pp	95.6%
KNN ($k = 5$)	96.3%	1.1 pp	95.6%
Gradient boosting	95.2%	1.5 pp	95.6%
Decision tree	91.6%	1.8 pp	91.2%

“pp” means percentage points. The table reproduces the supplied comparison. Logistic regression has the highest mean for these settings and folds. Its strong score is consistent with useful linear separation in the feature space. It does not prove perfect linear separability or that more flexible models are always worse on this dataset.

The holdout scores in this table are illustrative because the same holdout was already examined during the depth demonstration. In a clean comparison, use CV for choosing the candidate, freeze

the procedure, then reveal the final score. The later section on a fresh experiment gives that order explicitly.

5.2 The box plot of fold scores

```
pd.DataFrame(all_folds).plot(kind="box", figsize=(10, 4))
plt.ylabel("accuracy per fold")
plt.title("5-fold cross-validation scores")
plt.xticks(rotation=15)
plt.show()
```

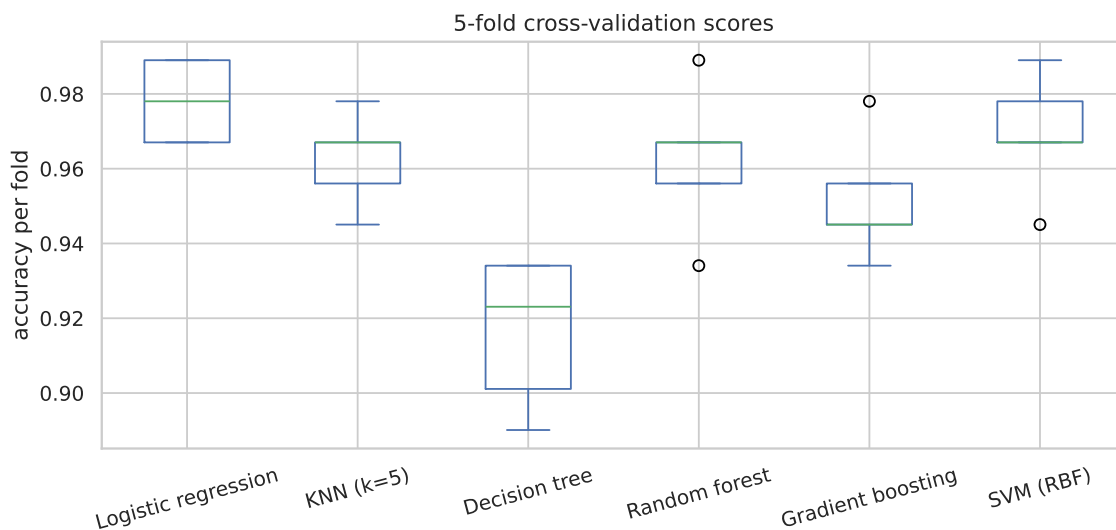


Figure 3: The five fold accuracies for each candidate, summarised as box plots.

`pd.DataFrame(all_folds)` creates one column per model and one row per fold. A box plot summarises the median and quartiles, with whiskers and possible outlying points. There are only five measurements per model, so treat the shape cautiously.

A higher median is useful, and spread tells us how consistent these particular fold scores were. A small apparent gap between models needs more evidence before being treated as a dependable advantage. Also consider inference time, memory, the cost of mistakes, and how the model will be used.

5.3 Reading the forest's importance plot

```
rf = models["Random forest"]
importance = pd.Series(rf.feature_importances_, index=bc.feature_names).sort_values(ascending=False)
importance.head(10).sort_values().plot(kind="barh")
plt.title("Random forest: top 10 most important measurements")
plt.show()
```

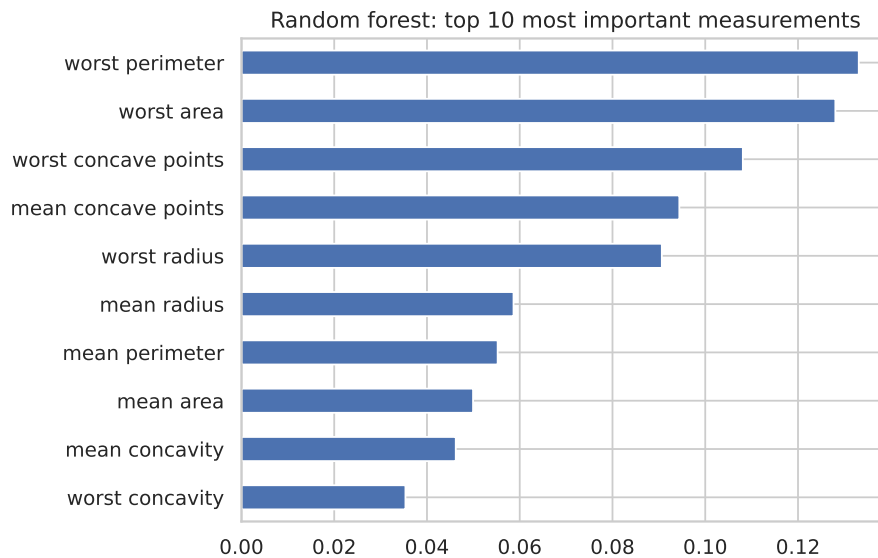


Figure 4: The ten highest impurity-based importances in the fitted forest.

`models["Random forest"]` retrieves the forest fitted in the comparison loop. Attributes ending in an underscore commonly hold quantities learned during fitting. `feature_importances_` contains one importance per input column. The Series associates each value with its feature name.

Sorting in descending order identifies the largest values. `head(10)` keeps ten. Sorting those ten in ascending order makes the horizontal chart place the largest near the top. Features such as worst perimeter, worst area, and worst concave points contribute strongly in this fit.

An importance near 0.13 represents about 13% of the normalised importance total. It does not mean the feature raises accuracy by 13 percentage points, nor does it give the direction of its association. A held-out permutation-importance analysis can provide another view by measuring how shuffling one feature changes an evaluation score.

5.4 The confusion matrix and class-specific recall

```
from sklearn.metrics import ConfusionMatrixDisplay, classification_report

best = models["Logistic regression"]
ConfusionMatrixDisplay.from_estimator(best, X_test, y_test,
                                     display_labels=["malignant", "benign"], cmap="Blues")
plt.title("Logistic regression on 114 final test tumours")
plt.show()
print(classification_report(y_test, best.predict(X_test), target_names=["malignant", "benign"]))
```

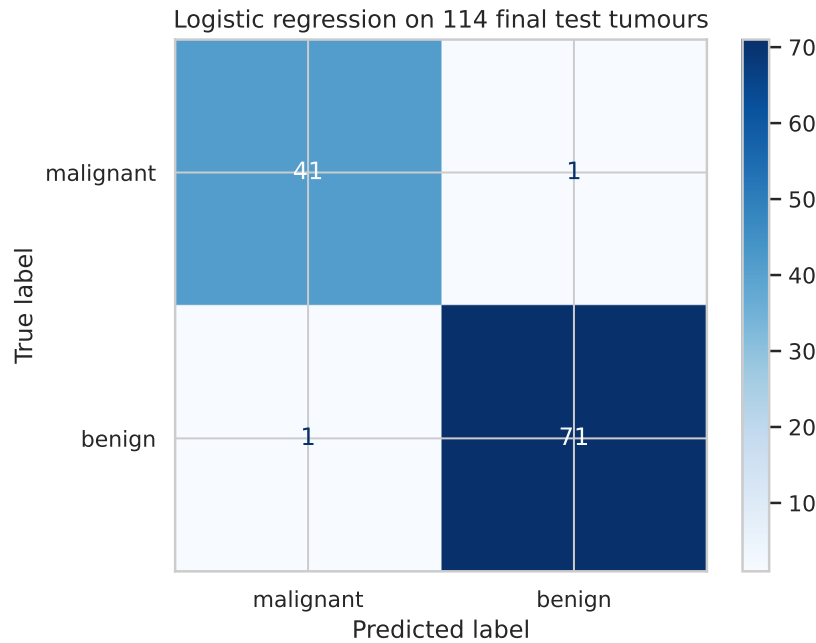


Figure 5: Rows show actual diagnosis categories; columns show predicted categories.

The matrix contains

	Predicted malignant	Predicted benign
Actual malignant	41	1
Actual benign	1	71

Forty-one malignant examples are identified correctly. One malignant example is predicted benign. One benign example is predicted malignant. Seventy-one benign examples are identified correctly. The total is $41 + 1 + 1 + 71 = 114$.

Taking malignant as the condition of interest, the missed malignant example is a false negative. The benign example predicted malignant is a false positive. The numerical encoding still uses zero for malignant, so a separate metric call must explicitly select `pos_label=0` if it uses a positive-class argument.

For this condition,

$$\text{recall} = \frac{41}{41 + 1} = 97.6\%, \quad \text{precision} = \frac{41}{41 + 1} = 97.6\%.$$

Recall asks how many actual malignant examples were detected. Precision asks how many malignant predictions were correct. Their numerical values happen to be equal here because there is one error of each type. Accuracy is $(41 + 71)/114 = 98.2\%$.

The F1 score is the harmonic mean $2PR/(P + R)$, where P is precision and R is recall. **support** is the number of actual examples in that class. **macro avg** averages the class metrics equally. **weighted avg** weights each class by its support. Rounded printed values can hide small differences.

The exercise demonstrates why the types of mistakes matter. Its small test sample provides a classroom evaluation of a dataset. Assessing a clinical system would require evidence about the intended population and use; these results alone do not establish that suitability.

6 Tuning settings with GridSearchCV

Material covered: Slide 16; Notebook 1, Part 4.

6.1 Parameters and hyperparameters

A learned parameter is adjusted by fitting, such as a logistic-regression coefficient or a tree's chosen threshold. A hyperparameter controls the learning procedure, such as maximum tree depth, the number of neighbours, or the SVM's C and γ .

A grid search lists possible hyperparameter values and tries every listed combination. The comparison uses validation folds within the training data. The best observed setting is then fitted on all the data supplied to the search.

```
svm_pipe = Pipeline([("scale", StandardScaler()), ("svm", SVC())])
param_grid = {"svm__C": [0.1, 1, 10, 100],
              "svm__gamma": ["scale", 0.001, 0.01, 0.1]}

grid = GridSearchCV(svm_pipe, param_grid, cv=cv)
grid.fit(X_train, y_train)

print("Best settings:", grid.best_params_)
print(f"Best CV accuracy: {grid.best_score_:.1%}")
print(f"Final test accuracy: {grid.score(X_test, y_test):.1%}")
```

Pipeline names the two steps explicitly: `scale` and `svm`. The keys in `param_grid` use a double underscore to reach a setting inside a step. Thus `svm__C` means the C argument of the step named `svm`.

There are four choices of C and four choices of γ , producing $4 \times 4 = 16$ combinations. Each receives five CV fits, so the search runs 80 fits for comparison. With the default refit behaviour, it then performs one additional fit of the best pipeline on all 455 training examples. That makes 81 fitting operations in total.

`gamma="scale"` asks SVC to calculate its default value from the feature count and the variance of the data it receives. Because scaling is inside the pipeline, this is calculated after scaling for each fit. The string is one of the grid choices alongside the three numerical values.

The verified best setting is $C = 10$ and $\gamma = 0.01$, with 97.6% mean CV accuracy. The held-out score is 98.2%. It improves on the default SVM's 96.9% CV mean in this comparison, while logistic regression still has the larger observed mean of 97.8%.

6.2 Reading the search table

```
table = pd.DataFrame(grid.cv_results_).pivot(index="param_svm__gamma", columns="param_svm__C",
      values="mean_test_score")
sns.heatmap(table, annot=True, fmt=".3f", cmap="Blues")
plt.title("CV accuracy for each (gamma, C)")
plt.xlabel("C"); plt.ylabel("gamma")
plt.show()
```

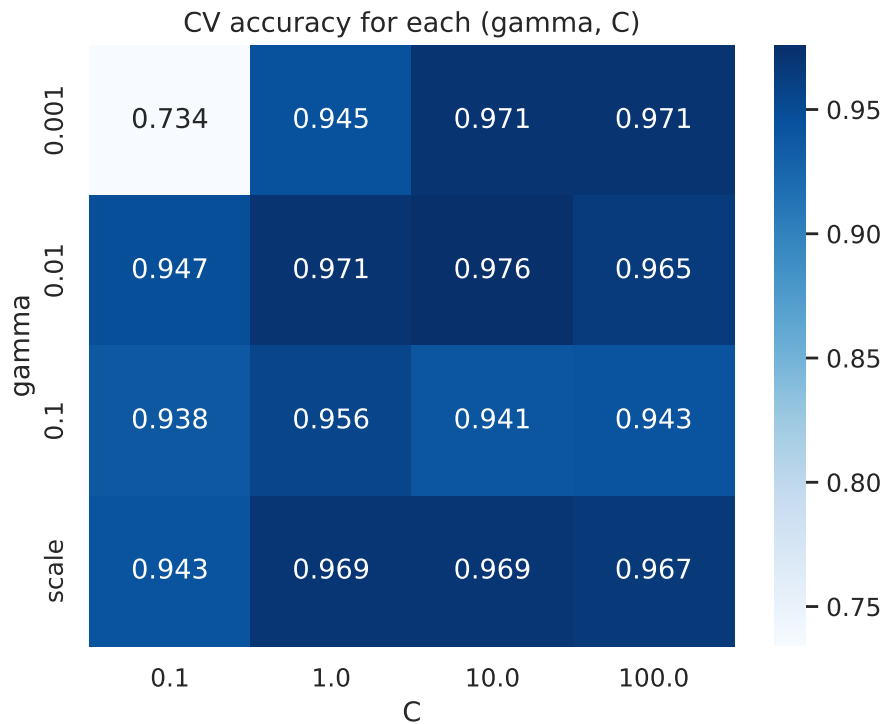


Figure 6: Validation accuracy for each tested SVM setting. The printed decimals are proportions.

`cv_results_` holds the search results. `pivot` arranges them so rows correspond to `gamma`, columns correspond to `C`, and cells contain the mean validation score. In this attribute, “test” refers to the validation fold used internally by the search.

`annot=True` writes a score in every square. `fmt=".3f"` displays three decimal places. A value of 0.976 means 97.6%. The poorly performing $C = 0.1$, $\gamma = 0.001$ combination obtains about 73.4%. Choosing settings can therefore matter substantially.

The highest score among many attempted settings is a selection result. It can be optimistic because the same CV results were used to choose it. A final independent evaluation checks the selected procedure. Nested CV is another option when a separate test set would use too much limited data: an outer validation loop evaluates a complete inner tuning process.

6.3 A fresh experiment with a genuinely reserved test set

The supplied notebook deliberately shows several holdout scores to make the lesson visible. Its later claim that the final set is still untouched does not match those earlier evaluations. For future work, follow this sequence from the start:

1. Reserve the final test examples before exploratory model comparisons.
2. Run the depth experiment, model comparison, preprocessing, and tuning using only the development portion and its validation folds.
3. Choose the model family, hyperparameters, features, and decision threshold using that development evidence.
4. Fit the selected full pipeline on all development examples.
5. Evaluate once on the final test set and report the result with its limitations.

If the final score prompts a redesign, those test examples have now informed development. A new independent evaluation is needed for the redesigned procedure. Re-running a split after looking at the entire dataset does not undo the knowledge already gained.

Here is the central code pattern for a fresh dataset, after the split has been created. It uses validation scores to inspect depth and keeps the final score until selection is complete:

```
for depth in [1, 2, 3, 4, 5, 6, None]:
    candidate = DecisionTreeClassifier(
        max_depth=depth, random_state=42)
    scores = cross_val_score(candidate, X_train, y_train, cv=cv)
    print(depth, scores.mean())

search = GridSearchCV(svm_pipe, param_grid, cv=cv)
search.fit(X_train, y_train)
# After all development decisions have been frozen:
final_score = search.score(X_test, y_test)
```

The code assumes the preceding named objects have been defined. This example illustrates the order of operations; it does not make the already-inspected classroom holdout unseen again.

7 Titanic: putting preparation inside the model pipeline

Material covered: Slides 17–18; Notebook 1, Part 5.

7.1 Why preprocessing belongs inside cross-validation

Suppose a validation passenger has a missing age. To fill it, we need an age summary learned from the training passengers. If the median was calculated from the complete table before splitting, the validation examples influenced that summary. This is a form of preprocessing leakage, even though the survival labels were not used directly.

A **transformer** learns or applies a change to input data. An imputer fills missing entries, a scaler changes feature units, and an encoder converts categories to numerical columns. A pipeline arranges these transformations before a final estimator. During a fit, each transformer learns from the data supplied to that fit. During prediction, it reuses those learned quantities.

For example, standardisation uses

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}.$$

The training mean μ_{train} gives the centre. The training standard deviation σ_{train} gives the scale. A validation passenger's age is transformed using these same numbers. Computing a separate mean for the validation data would give it a different coordinate system.

7.2 Building the two preprocessing branches

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder

titanic = sns.load_dataset("titanic")
titanic["family_size"] = titanic["sibsp"] + titanic["parch"] + 1

num_cols = ["pclass", "age", "sibsp", "parch", "fare", "family_size"]
cat_cols = ["sex", "embarked"]
Xt = titanic[num_cols + cat_cols]      # raw data: missing values and text included!
yt = titanic["survived"]

num_steps = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())
cat_steps = make_pipeline(SimpleImputer(strategy="most_frequent"), OneHotEncoder(handle_unknown="ignore"))
prep = ColumnTransformer([("num", num_steps, num_cols), ("cat", cat_steps, cat_cols)])

titanic_models = {
    "Logistic regression": LogisticRegression(max_iter=1000),
    "SVM (RBF)": SVC(),
    "Random forest": RandomForestClassifier(n_estimators=200, max_depth=6, random_state=42),
    "Gradient boosting": GradientBoostingClassifier(random_state=42),
}

cv_t = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for name, m in titanic_models.items():
    pipe = make_pipeline(prepare, m)
    s = cross_val_score(pipe, Xt, yt, cv=cv_t)
    print(f"{name:20s} {s.mean():.1%} +/- {s.std():.1%}")
```

`sns.load_dataset("titanic")` loads the passenger table. The new `family_size` column adds siblings/spouses, parents/children, and the passenger themselves. This is a fixed row-by-row calcula-

tion, so it does not estimate a statistic from other passengers. Applying it before CV is appropriate here.

The numerical columns include class, age, relatives, fare, and family size. Class is represented numerically in this notebook, so logistic regression applies a numerical effect to its values. Encoding class categorically would represent it differently and would be another modelling choice to evaluate.

The categorical columns are sex and port of embarkation. `num_cols + cat_cols` joins two Python lists of column names. Selecting those columns creates `Xt`; selecting survival creates `yt`. Variables with different names help keep this task separate from the breast cancer arrays.

`SimpleImputer(strategy="median")` fills a missing numerical value with that column's training median. The numerical branch then applies `StandardScaler`. The categorical branch fills missing entries with the most frequent training category, then applies one-hot encoding.

For a category such as embarkation port, one-hot encoding creates separate indicator columns. A passenger receives a one in the column for their port and zeros in the other port columns. This represents categories without assigning an arbitrary increasing numerical scale.

`handle_unknown="ignore"` allows a future category that was absent during training. Its indicator block is represented by zeros for the learned categories. This keeps prediction running, but it also means the model has no fitted category-specific effect for that unseen value.

`ColumnTransformer` sends selected columns through different branches and combines their numerical outputs. Each tuple contains a branch name, its transformer, and the column list. The output is then passed to the classifier by the outer pipeline.

7.3 What the cross-validation loop actually fits

The Titanic dictionary contains four classifiers. The forest has 200 trees and a maximum depth of six. The numerical scaler stays in the shared preprocessing design even for the trees; that is harmless for this ordinary scaling transformation.

In each of the five CV runs, a fresh copy of the complete pipeline learns its imputation values, category vocabulary, scaling values, and model from that fold's training passengers. Validation passengers pass through those already-fitted steps. This handles the preprocessing leakage identified in the earlier notebook.

A pipeline still depends on appropriate inputs and splits. A feature copied from the answer, duplicate records across folds, or related family members split across folds can create other problems. The pipeline cannot detect those automatically.

The results are:

Model	Mean CV accuracy	Fold spread
Logistic regression	79.4%	1.7 pp
SVM (RBF)	82.6%	1.8 pp
Random forest	82.7%	1.8 pp
Gradient boosting	83.2%	1.6 pp

These are CV results across the selected Titanic data. This part of the notebook does not reserve an additional final test set. The 83.2% is therefore the observed cross-validation estimate for that pipeline, with model selection based on the same comparison.

An **interaction** means the relationship involving one feature depends on another feature. For example, the association between sex and survival may vary across passenger classes. A tree can represent this through successive splits. Logistic regression with only the supplied additive features

has more limited ways to describe it. Explicit interaction features could extend that model. The scores alone do not prove which specific interaction caused the difference.

7.4 Predicting from raw passenger rows

```
final = make_pipeline(prepare, GradientBoostingClassifier(random_state=42)).fit(Xt, yt)

new_passengers = pd.DataFrame([
    {"pclass": 1, "age": 17, "sibsp": 1, "parch": 1, "fare": 100.0, "family_size": 3, "sex":
     "female", "embarked": "S"},
    {"pclass": 3, "age": np.nan, "sibsp": 0, "parch": 0, "fare": 7.9, "family_size": 1, "sex":
     "male", "embarked": "S"},
], index=["1st-class girl, 17", "3rd-class man, age unknown"])

for name, p in zip(new_passengers.index, final.predict_proba(new_passengers)[: , 1]):
    print(f"{name}: {p:.0%} chance of survival")
```

The final pipeline is fitted on all selected Titanic rows. This stores the preparation steps and classifier together. The new table supplies the same required input columns, including a missing age represented by `np.nan`.

The first passenger has first-class travel, age 17, one sibling/spouse, one parent/child, fare 100, family size three, sex female, and embarkation S. The second has third-class travel, an unknown age, no recorded relatives aboard, fare 7.9, family size one, sex male, and embarkation S.

`predict_proba` returns one probability column per class. Here the class order is zero then one, so `[:, 1]` selects the model's estimated survival probability. The colon takes all rows. `zip` pairs each passenger name with its prediction for printing.

The outputs are about 96% and 8%. They are model estimates based on this historical dataset and representation. Their calibration has not been established by the notebook. A reported 96% does not guarantee the outcome of an individual case.

The preprocessing pipeline expects `family_size` to be supplied because it was created before the pipeline. It does not automatically calculate that column for a new passenger. A production implementation could add a feature-building transformer so that the same calculation is applied consistently at training and prediction time.

8 Notebook 1 exercises, with explanations

Material covered: Notebook 1 exercises and solution cells; homework in slide 34.

8.1 Exercise 1: tune a random forest

```
# Exercise 1
rf_grid = GridSearchCV(RandomForestClassifier(random_state=42),
                       {"n_estimators": [100, 300], "max_depth": [None, 5, 10]}, cv=cv)
rf_grid.fit(X_train, y_train)
print("Best:", rf_grid.best_params_, f"CV {rf_grid.best_score_:.1%}")
print(f"Logistic regression CV was {showdown.loc['Logistic regression', 'CV mean']:.1%} - still
      the winner.")
```

There are two choices for the number of trees and three for depth. This produces six combinations. With five folds, there are 30 CV fits, plus one refit of the selected configuration.

The best observed settings are 300 trees and `max_depth=None`, with about 96.5% mean CV accuracy. The logistic-regression mean is 97.8%. The code retrieves that earlier result with `showdown.loc`, which selects a named row and column.

An unrestricted individual tree performed poorly earlier, yet an unrestricted tree inside a forest can still be useful. The forest combines many varied fits. Its behaviour depends on the ensemble as a whole. Choosing the depth from these results is appropriate for this development experiment; it is not a universal setting for every forest.

The solution's final printed sentence is hard-coded to say logistic regression remains the winner. If you change the data, grid, or folds, compare the calculated values before repeating that conclusion.

8.2 Exercise 2: choose the KNN neighbourhood

```
# Exercise 2
for k in [1, 3, 5, 9, 15, 25]:
    s = cross_val_score(make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=k)),
                        X_train, y_train, cv=cv)
    print(f"k={k:2d}: {s.mean():.1%} +/- {s.std():.1%}")
```

The loop builds a new scaled KNN pipeline for each candidate k . It evaluates every candidate using the same folds. The results are:

Neighbours	CV mean	Fold spread
1	94.7%	1.9 pp
3	96.3%	0.9 pp
5	96.3%	1.1 pp
9	96.5%	1.5 pp
15	96.0%	1.6 pp
25	95.2%	1.8 pp

Nine neighbours gives the highest mean among these choices. At one neighbour, a single unusual nearby example can determine the result. At 25, the neighbourhood may include examples from a broader region and blur a useful boundary. The differences around the middle of the table are small, so the ranking should be read with the fold variability in mind.

8.3 Exercise 3: optional XGBoost comparison

```
# Exercise 3 (optional)
try:
    from xgboost import XGBClassifier
    s = cross_val_score(XGBClassifier(n_estimators=200, eval_metric="logloss"), X_train, y_train,
                        cv=cv)
    print(f"XGBoost CV: {s.mean():.1%} +/- {s.std():.1%}")
except ImportError:
    print("XGBoost not installed - run: !pip install xgboost")
```

The `try` block attempts to import a separate package. If it is unavailable, `except ImportError` prints the installation hint and allows the notebook to finish. That missing-package branch was reached in the verification environment used for these notes.

If installed, the code fits a 200-tree XGBoost classifier in the same five folds. Its `eval_metric` setting selects log loss for XGBoost; the surrounding `cross_val_score` call still reports the classifier's default accuracy because its `scoring` argument is omitted.

The speaker notes report 96.7% CV accuracy with 1.4 percentage points of spread for their run. That optional number was not reproduced here. Package versions and default settings should be recorded when making the comparison. In a notebook, the suggested `!pip install xgboost` line asks the shell to install the package; it is not ordinary Python syntax for a script.

9 Notebook 2: finding customer groups with K-Means

Material covered: Slides 19–22 and 27; Notebook 2, Parts 1–2.

9.1 What can we learn without target labels?

In supervised learning, an answer column tells the model what to predict. Unsupervised learning studies structure in the input features. Clustering looks for groups, dimensionality reduction creates a smaller representation, and anomaly detection looks for unusual examples.

Even without target labels, the analyst makes choices: which columns to include, how to measure distance, whether to scale features, and what result would be useful. Those choices shape the patterns found.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")

from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
print("Ready!")
```

This setup repeats the numerical, table, plotting, and warning imports because Notebook 2 can run independently. `KMeans` and `AgglomerativeClustering` provide two clustering procedures. `silhouette_score` evaluates a particular kind of compactness and separation.

9.2 Loading the customer table

```
url = "https://raw.githubusercontent.com/tirthajyoti/Machine-Learning-with-Python/master/
      Datasets/Mall_Customers.csv"
mall = pd.read_csv(url)
mall = mall.rename(columns={"Annual Income (k$)": "income", "Spending Score (1-100)": "spending"
})
print(mall.shape)
mall.head()
```

`pd.read_csv(url)` reads the CSV at the supplied source address. The table initially contains 200 rows and five columns: CustomerID, Gender, Age, annual income, and spending score. `rename` shortens two column names using a dictionary mapping old names to new names.

Annual income is expressed in thousands of dollars. An income value of 15 therefore represents \$15,000 per year in this dataset. Spending is a score from 1 to 100. The notebook does not supply its detailed construction, so interpret it as the provided score. It is not a dollar amount or a verified measure of a person's motivations.

CustomerID identifies a row. Treating it as a distance feature would make nearby record numbers seem similar for an arbitrary reason. The first clustering uses only income and spending. Age and gender are available in the table but do not enter this initial fit.

```
sns.scatterplot(data=mall, x="income", y="spending", s=60)
plt.title("Mall customers (no labels)")
plt.show()
```

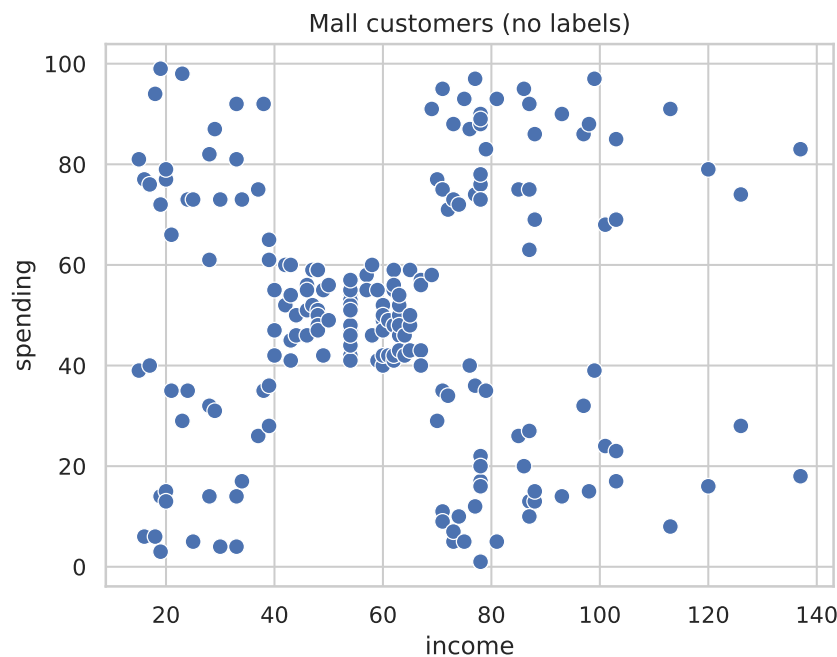


Figure 7: The two selected customer features before assigning cluster colours.

`s=60` sets the marker size. Before running K-Means, look for dense regions and gaps. Five broad regions are plausible here: a central group and four groups toward the corners. Human inspection gives a starting hypothesis that we can examine with numerical summaries.

9.3 Distances and centroids

A **centroid** is the coordinate-wise mean of the points in a cluster. If three customers have income/spending coordinates (20, 80), (30, 70), and (25, 90), their centroid is

$$\left(\frac{20 + 30 + 25}{3}, \frac{80 + 70 + 90}{3} \right) = (25, 80).$$

It need not correspond to an actual customer. It represents the average location of the group in the selected feature space.

Suppose a customer is at (30, 70) and two centres are (25, 80) and (85, 20). The squared distances are

$$(30 - 25)^2 + (70 - 80)^2 = 125,$$

$$(30 - 85)^2 + (70 - 20)^2 = 5525.$$

The first centre is closer. K-Means assigns the customer to that centre's group in its assignment step.

9.4 The K-Means learning loop

With a chosen number of clusters k , the procedure alternates between two operations: assign each point to its closest centre, then move each centre to the mean of its assigned points. It repeats until the changes become small enough or an iteration limit is reached.

The objective is to reduce **inertia**, the sum of squared distances from points to their assigned centres:

$$J = \sum_{i=1}^n \|x_i - \mu_{c(i)}\|^2.$$

x_i is customer i 's feature vector. $c(i)$ is the index of its cluster, and $\mu_{c(i)}$ is that cluster's centroid. n is the number of customers. The double bars denote Euclidean distance before it is squared.

Initial centres matter because the procedure can settle at a local solution. The slide illustrates random starting centres. The notebook uses the default K-Means++ initialisation, which chooses spread-out starting centres using a probabilistic procedure. `n_init=10` runs ten initialisations and keeps the result with the lowest inertia.

```
X = mall[["income", "spending"]].values

kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
mall["cluster"] = kmeans.fit_predict(X)

sns.scatterplot(data=mall, x="income", y="spending", hue="cluster", palette="tab10", s=60)
centres = kmeans.cluster_centers_
plt.scatter(centres[:, 0], centres[:, 1], c="black", marker="X", s=250, label="centres")
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
plt.title("K-Means with k = 5")
plt.show()
```

`mall[["income", "spending"]]` selects a two-column table. The double brackets select a list of columns. `.values` converts it to a NumPy array with shape (200, 2).

`n_clusters=5` requests five groups. `fit_predict(X)` learns the centres and returns the assigned group number for every customer. No target array is passed. Those numbers become a new column for plotting and profiling.

`cluster_centers_` has shape (5,2): five centres with two coordinates each. `centres[:, 0]` selects their incomes, and `centres[:, 1]` selects spending. Large black X markers show the centres. The legend is moved outside the axes with `bbox_to_anchor` so it covers fewer points.

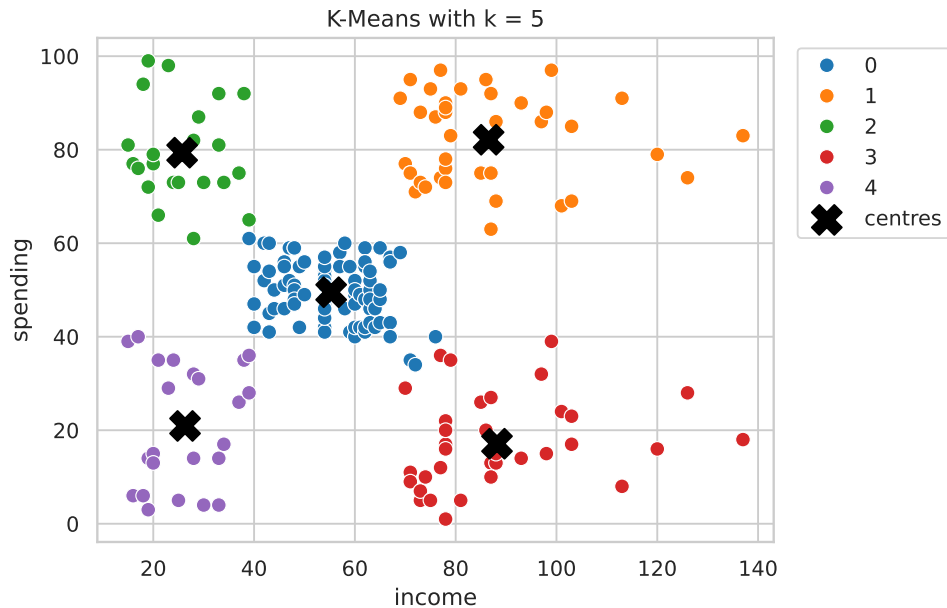


Figure 8: Five K-Means groups with their fitted centres.

The group numbers zero through four are arbitrary identifiers. Another run can assign different numbers to the same geometric groups. Compare their locations and profiles before deciding that the result changed.

9.5 Why feature scales matter here

The notebook leaves income and spending unscaled because their ranges are roughly comparable. This is a modelling choice. One income unit represents a thousand dollars, while one spending unit represents one score point. Euclidean distance implicitly gives those numerical units equal weight.

If income were expressed in individual dollars, its squared differences would dominate. Scaling prevents an accidental unit change from overwhelming the result, but also changes the relative weighting of features. Even similar-looking ranges do not prove that equal weighting is appropriate. The age exercise later uses standardisation explicitly.

K-Means is especially suited to compact groups under squared Euclidean distance. Unequal densities, elongated shapes, outliers, and irrelevant features can make its groups less useful. It always assigns each example to one of the requested clusters, even when the data contains weak cluster structure.

9.6 Describing the groups in ordinary language

```

profile = mall.groupby("cluster").agg(customers=("income", "size"), income=("income", "mean"),
                                     spending=("spending", "mean"), age=("Age", "mean")).round
    (1)

def name_group(row):
    inc = "High income" if row.income > 70 else ("Low income" if row.income < 40 else "Middle
income")
    spd = "high spending" if row.spending > 60 else ("low spending" if row.spending < 40 else "
middle spending")
    return f"{inc}, {spd}"

profile["description"] = profile.apply(name_group, axis=1)
profile

```

`groupby("cluster")` collects rows by assigned group. Named aggregations specify both an output name and a calculation. `customers=("income", "size")` counts group rows. The other entries calculate mean income, spending, and age. `round(1)` formats the summary to one decimal place.

The function `name_group` receives one profile row. It calls income above 70 high and below 40 low. Spending above 60 is high and below 40 is low. Values between the thresholds receive the middle label. These thresholds are human-written description rules. K-Means did not learn those words or cut-offs.

`apply(name_group, axis=1)` runs the function once for each profile row. The resulting strings become the description column.

Group	Count	Income	Spending	Age	Description
0	81	55.3	49.5	42.7	Middle income, middle spending
1	39	86.5	82.1	32.7	High income, high spending
2	22	25.7	79.4	25.3	Low income, high spending
3	35	88.2	17.1	41.1	High income, low spending
4	23	26.3	20.9	45.2	Low income, low spending

The counts sum to 200. Age is a description added after fitting; it did not influence this clustering. Group one's average age of 32.7 does not mean every person in it is approximately 33.

The slides use memorable names such as “target customers”, “careful earners”, and “young impulse buyers”. The measured evidence supports the income, spending, and age summaries. Claims about carefulness or impulsiveness would require additional evidence about behaviour and intent. Descriptive names make the limits clearer.

A bank could similarly explore groups using transaction amounts and frequencies. A group might motivate a product hypothesis or a closer review of unusual activity. Whether that group is useful must be checked against the actual business question and additional evidence.

9.7 Choosing k: elbow and silhouette

```

ks = range(2, 11)
inertias, silhouettes = [], []
for k in ks:
    km = KMeans(n_clusters=k, random_state=42, n_init=10).fit(X)
    inertias.append(km.inertia_)
    silhouettes.append(silhouette_score(X, km.labels_))

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(list(ks), inertias, marker="o"); ax[0].set_title("Elbow: inertia"); ax[0].set_xlabel("k")
ax[1].plot(list(ks), silhouettes, marker="o", color="orange"); ax[1].set_title("Silhouette score"); ax[1].set_xlabel("k")
plt.show()

best_k = list(ks)[int(np.argmax(silhouettes))]
print(f"Best silhouette: k = {best_k} ({max(silhouettes):.3f})")

```

`range(2, 11)` tries cluster counts two through ten. Every iteration fits a new K-Means model. `inertia_` records its within-group squared-distance total. The silhouette function compares within-group and between-group distances for the assigned labels.

Adding centres gives the optimal K-Means objective more flexibility, so its minimum possible inertia cannot increase as k grows. An actual finite run can occasionally settle in a poorer local solution. At $k = 200$, each of these 200 observations could have its own centre and zero distance. Inertia alone therefore cannot identify a useful group count.

An **elbow** is a point where additional clusters give noticeably smaller improvements. It is a visual judgement and can be unclear. The silhouette score gives another perspective. For one point, let a be its average distance to other points in its own cluster. Let b be its smallest average distance to another cluster. Then

$$s = \frac{b - a}{\max(a, b)}.$$

If $a = 2$ and $b = 8$, then $s = (8 - 2)/8 = 0.75$, indicating a relatively compact and separated assignment. A value near zero indicates overlap at a boundary. A negative value means the point is, on average, closer to another cluster. The reported silhouette is the average over the points, with a convention of zero for a singleton cluster.

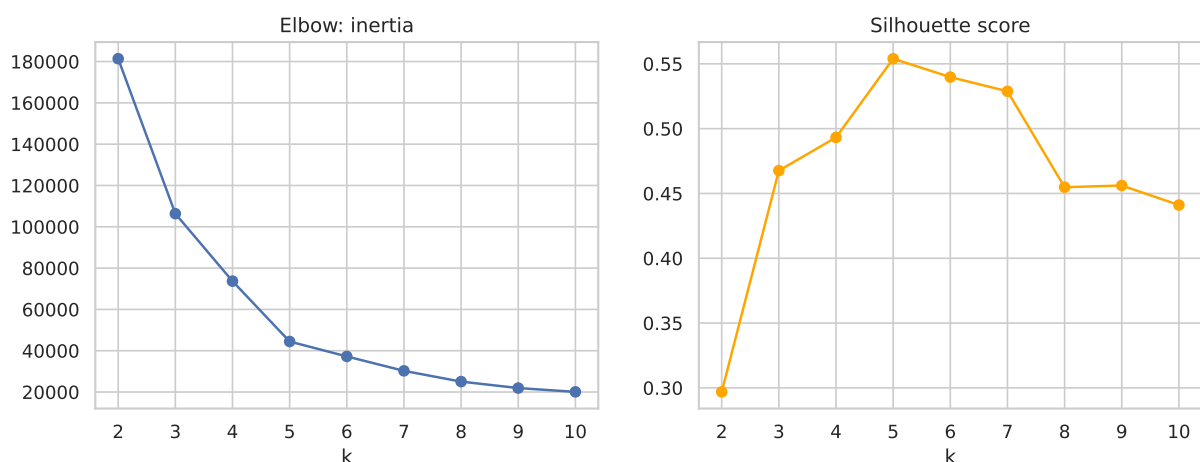


Figure 9: Inertia and average silhouette for the tested cluster counts.

`np.argmax(silhouettes)` finds the position of the largest score, and indexing `list(ks)` recovers the corresponding k . Here the best tested value is five with silhouette 0.554. This supports five as a useful choice under the current features and distance calculation. It is not classification accuracy, and it does not prove that the population has exactly five natural customer types.

10 Hierarchical clustering and the dendrogram

Material covered: Slide 23; Notebook 2, Part 3.

10.1 Building groups through successive merges

Agglomerative hierarchical clustering begins with each observation in its own group. It merges groups repeatedly according to a linkage rule. A hierarchy records these merges, so you can inspect groupings at several levels of detail.

The notebook uses **Ward linkage**. It chooses merges that produce a small increase in the total within-cluster sum of squares. Calling it “merge the closest groups” is a helpful first description, but the linkage rule determines exactly what closeness means. Ward’s rule relates to compactness and uses Euclidean geometry.

```
from scipy.cluster.hierarchy import dendrogram, linkage

plt.figure(figsize=(12, 5))
dendrogram(linkage(X, method="ward"), no_labels=True, color_threshold=150)
plt.axhline(150, color="orange", linestyle="--")
plt.title("Dendrogram of 200 mall customers (cut at the dashed line)")
plt.ylabel("distance when groups merge")
plt.show()
```

`linkage(X, method="ward")` computes the sequence of merges. `dendrogram` draws that sequence. With `no_labels=True`, the chart omits 200 individual leaf labels so that the structure is easier to see. The colour threshold and dashed horizontal line both use a merge height of 150.

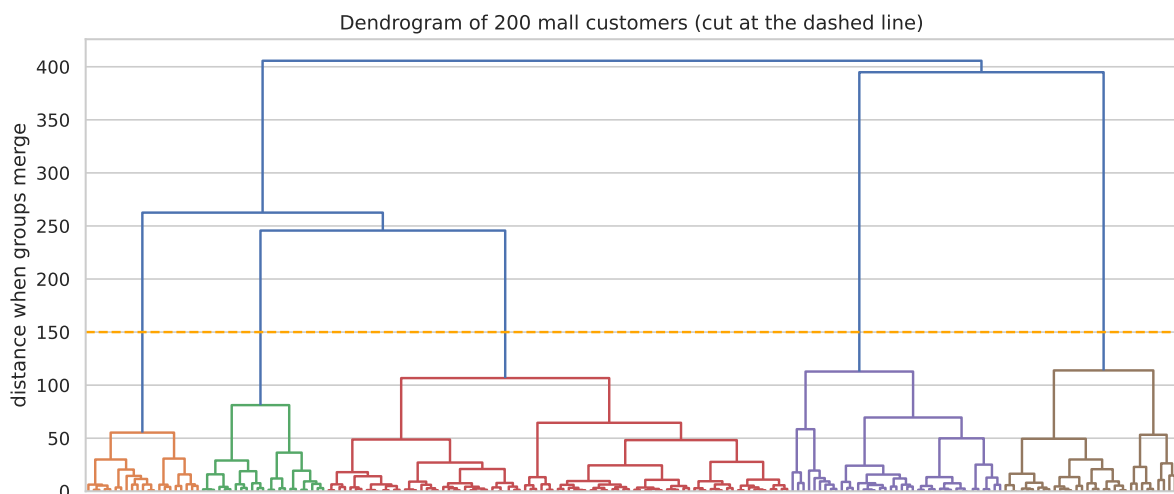


Figure 10: Ward-linkage dendrogram for the customer features.

Read from the bottom upward. Leaves represent customers. A join combines two existing groups. The height of a join describes their dissimilarity under the linkage calculation. The horizontal order

organises the tree for display; horizontal spacing is not an income or spending axis.

A horizontal cut forms groups from the branches below it. In this actual notebook dendrogram, a cut at 150 gives five groups. The slide's simplified picture illustrates four groups. They show the same principle with different drawings.

You can build the hierarchy before choosing a final number of clusters. You still need to choose a cut or grouping level for a practical segmentation. Hierarchical methods also become more expensive as the number of observations grows.

10.2 Comparing two cluster assignments

```
from sklearn.metrics import adjusted_rand_score

agg = AgglomerativeClustering(n_clusters=5)
mall["hier"] = agg.fit_predict(X)

print(f"Agreement with K-Means (1 = identical): {adjusted_rand_score(mall['cluster'], mall['hier']):.2f}")
pd.crosstab(mall["cluster"], mall["hier"], rownames=["K-Means"], colnames=["Hierarchical"])
```

`AgglomerativeClustering(n_clusters=5)` requests five groups from the agglomerative procedure. The default linkage here is Ward. Its assignments are stored in `mall["hier"]`.

Adjusted Rand index, or ARI, compares pairs of examples: do the two clustering methods agree about which pairs belong together? It adjusts this agreement for chance under its reference model. A value of one means identical groupings up to renaming, zero corresponds to chance-level agreement under that model, and negative values are possible.

The observed ARI is about 0.94. This is strong agreement between these two assignments. It is not 94% classification accuracy, because there are no supplied true customer-type labels.

`pd.crosstab` counts how the labels line up:

K-Means group	Hierarchical group				
	0	1	2	3	4
0	0	81	0	0	0
1	0	0	39	0	0
2	0	1	0	21	0
3	32	3	0	0	0
4	0	0	0	0	23

For example, all 81 members of K-Means group zero appear in hierarchical group one. The different numbers simply name the same set. One member of K-Means group two and three members of group three join the large central hierarchical group.

Agreement makes the grouping less dependent on one particular algorithm. Both methods still use the same two features and related compactness assumptions. Their agreement provides evidence of stable structure under those choices; it does not independently verify the business meaning of the groups.

11 PCA: representing many measurements with fewer numbers

Material covered: Slides 24–25; Notebook 2, Part 4.

11.1 Turning an image into a feature vector

```
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA

digits = load_digits()
print(digits.data.shape)

fig, axes = plt.subplots(1, 8, figsize=(12, 2))
for ax, img, label in zip(axes, digits.images, digits.target):
    ax.imshow(img, cmap="gray_r"); ax.set_title(label); ax.axis("off")
plt.show()
```

`load_digits()` supplies 1,797 small handwritten digit images, their flattened pixel measurements, and their known digit labels. Each image has eight rows and eight columns, giving $8 \times 8 = 64$ pixels. The pixel intensities in this dataset range from zero to sixteen.

`digits.images` holds the 8×8 image arrays. `digits.data` holds a table of shape $(1797, 64)$: one row per image and one column per pixel position. Flattening changes the arrangement of the same measurements; it does not create new information. `digits.target` contains categories zero through nine.

`plt.subplots(1, 8)` creates eight axes in one row. `zip` pairs each axis with an image and label. Since the axis list has eight entries, the loop displays the first eight images. `imshow` draws the pixel values, `gray_r` chooses reversed greyscale, and `axis("off")` removes coordinate ticks.

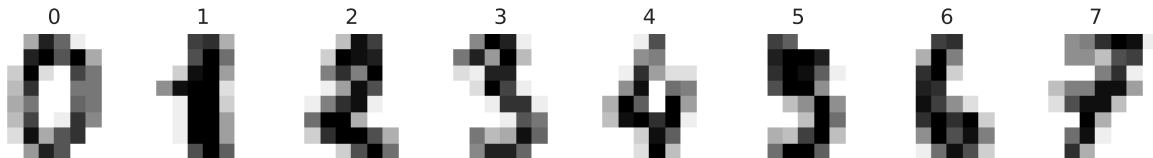


Figure 11: Examples from the digit dataset. Each image supplies 64 numerical input values.

A point with 64 coordinates cannot be shown directly on an ordinary two-axis scatter plot. We need a rule that maps those 64 coordinates to two coordinates if we want such a picture.

11.2 What a principal component means

Principal component analysis, or PCA, constructs new coordinates as weighted combinations of the original features. It first centres the columns. It then finds directions along which the centred data varies most. The first component has the largest variance. The next has the largest remaining variance subject to being perpendicular to the first, and so on.

Imagine two features that increase together. A diagonal direction running through their cloud of points may capture much of their movement. The perpendicular direction may capture only small deviations from that trend. Keeping the diagonal coordinate gives a compact approximation.

For one component, an example's new coordinate is

$$z_1 = w_{11}(x_1 - \bar{x}_1) + \dots + w_{1p}(x_p - \bar{x}_p).$$

The x_j values are that example's original features. $\bar{x}_j$ is the fitted mean of feature j . The w_{1j} values describe the learned direction. The result z_1 is one number for that example. A second direction supplies z_2 .

For a simple two-feature illustration, suppose the data is centred and the main direction has weights $(1/\sqrt{2}, 1/\sqrt{2})$. A point $(2, 2)$ gets coordinate $4/\sqrt{2} \approx 2.83$ along that direction. A perpendicular direction with weights $(1/\sqrt{2}, -1/\sqrt{2})$ gives coordinate zero. Its equal feature values lie entirely along the first direction in this illustration.

PCA selects directions using the features. Labels play no role in this fitting step. A direction with large variance may help classification, but the PCA objective does not directly optimise class separation.

11.3 The two-component digit plot

```
pca2 = PCA(n_components=2)
coords = pca2.fit_transform(digits.data)      # note: no labels used!

plt.figure(figsize=(9, 7))
sc = plt.scatter(coords[:, 0], coords[:, 1], c=digits.target, cmap="tab10", s=12)
plt.colorbar(sc, label="digit (only used for colour)")
plt.title("1,797 digit images squeezed from 64 numbers to 2")
plt.show()

print(f"Information kept by 2 components: {pca2.explained_variance_ratio_.sum():.1%}")
```

`PCA(n_components=2)` requests two new features. `fit_transform` learns the directions and calculates the new coordinates for the same images. `coords` has shape $(1797, 2)$.

The scatter uses the first and second coordinate columns for its axes. `c=digits.target` colours points using known digit labels after the projection has been fitted. This helps us examine whether the projected arrangement corresponds to the categories. It does not mean labels entered the PCA fit.

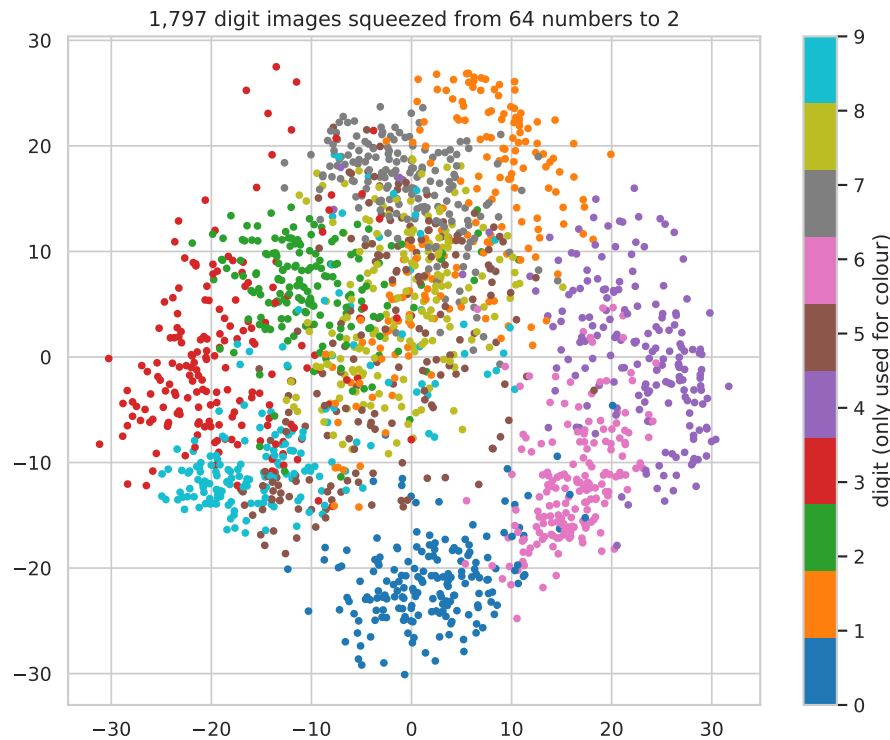


Figure 12: The digit images projected onto two principal components. Colours identify known digit labels for interpretation.

Some digits, including many zeros, fours, and sixes, occupy distinctive regions. Other categories, including threes, fives, eights, and nines, overlap substantially. Projection can place different original images near each other because 62 directions have been discarded. The plot provides a useful view of structure while losing detail.

`explained_variance_ratio_` contains the fraction of total fitted variance associated with each retained component. Adding the two entries gives about 28.5%. In these notes, “variance retained” is the precise interpretation of the notebook’s printed “information kept”. That percentage is not classification accuracy or a count of correctly recognised images.

11.4 Choosing a number of components

```
pca_all = PCA().fit(digits.data)
cumulative = np.cumsum(pca_all.explained_variance_ratio_)
plt.plot(range(1, 65), cumulative, marker=".")
plt.axhline(0.9, color="orange", linestyle="--")
plt.xlabel("number of components"); plt.ylabel("information kept")
plt.title("Cumulative explained variance")
plt.show()
print("Components needed for 90%:", int(np.argmax(cumulative >= 0.9)) + 1)
```

`PCA()` fits all available components here. `np.cumsum` forms cumulative sums: first the variance fraction of component one, then components one and two, then one through three, and so forth.

For example, if the first three ratios were 0.40, 0.25, and 0.15, the cumulative values would be 0.40, 0.65, and 0.80. Keeping three would retain 80% of variance in that example. The notebook does the same calculation for all 64 digit features.

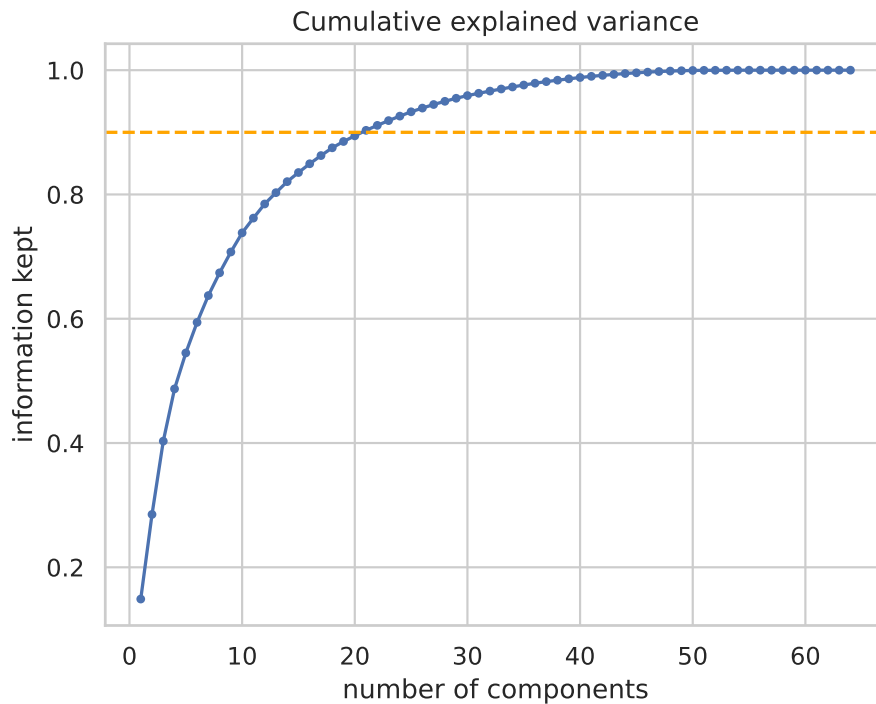


Figure 13: Cumulative explained variance for PCA fitted to the raw digit features.

The expression `cumulative >= 0.9` makes a Boolean array. `np.argmax` returns the first position containing the maximum Boolean value, which is `True` once the threshold is reached. Adding one changes a zero-based position into a component count. Here the threshold is reached at 21 components.

This trick assumes the threshold is reached. For other code, check that at least one entry is `True` before using the result. A Boolean array containing only `False` values would otherwise give a misleading first position.

11.5 Scaling changes the PCA problem

The preceding visualisations apply PCA directly to raw digit intensities. PCA centres features automatically, but does not standardise every feature to unit variance. Since the raw pixels use the same intensity scale, the visualisation is a reasonable exploratory choice.

The classification experiment below applies `StandardScaler` before PCA. This gives each nonconstant pixel column unit training variance, including pixels with relatively little raw variation. The resulting directions and variance ratios can differ from raw-pixel PCA. Therefore the earlier 28.5% and 21-component results should not be silently attached to the scaled classification pipeline.

11.6 How dimensionality affects classification

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

Xd_train, Xd_test, yd_train, yd_test = train_test_split(
    digits.data, digits.target, test_size=0.2, random_state=42, stratify=digits.target)

for n in [2, 10, 20]:
    m = make_pipeline(StandardScaler(), PCA(n_components=n), LogisticRegression(max_iter=5000)).
        fit(Xd_train, yd_train)
    print(f"{n:2d} components: accuracy {m.score(Xd_test, yd_test):.1%}")
m = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000)).fit(Xd_train, yd_train)
print(f"all 64 pixels: accuracy {m.score(Xd_test, yd_test):.1%}")
```

The stratified split reserves 360 digit images for evaluation and keeps 1,437 for training. In each pipeline, the scaler and PCA learn from the training images only. The logistic-regression classifier then learns from the transformed training features and known training labels.

For a new test image, the fitted scaler transforms its pixels, the fitted PCA projects it, and the fitted classifier predicts its digit. This preserves a consistent representation. Fitting PCA to all images before this predictive evaluation would let the test inputs influence the learned transformation.

Representation supplied to logistic regression	Test accuracy
2 scaled-data principal components	52.5%
10 scaled-data principal components	87.8%
20 scaled-data principal components	94.2%
All 64 scaled pixels	97.2%

Two components give a convenient picture, but discard many distinctions relevant to digit recognition. Twenty components recover much of the observed performance. The gap from all 64 pixels is about three percentage points in this run.

Reducing 64 features to 20 reduces the number of coordinates passed to the classifier by a factor of $64/20 = 3.2$. It does not automatically make the complete deployed model 3.2 times smaller. The PCA directions, scaling values, and projection computation also require storage and work. For a device such as an ESP32, measure the complete implementation's memory, latency, and accuracy.

The four held-out scores make a useful teaching comparison. If they guide a choice of component count, they are acting as validation evidence. A final performance report should then use an additional independent evaluation or a suitable nested procedure.

11.7 When the PCA representation helps

PCA can produce visualisations, reduce the number of downstream features, and remove variation associated with small components. Sometimes that discarded variation is mostly noise; sometimes it contains information important to the task. The result depends on the dataset and the goal.

PCA components mix original features, so they are often less directly interpretable than named measurements. A component sign can also flip between valid implementations without changing the represented geometry. A horizontally mirrored PCA chart can therefore describe the same structure.

12 Detecting unusual motor readings

Material covered: Slide 26; Notebook 2, Part 5.

12.1 Building the simulated example

```
rng = np.random.RandomState(42)
normal = np.column_stack([rng.normal(50, 3, 480), rng.normal(2.0, 0.4, 480)])
faulty = np.column_stack([rng.normal(58, 3, 20), rng.normal(3.2, 0.5, 20)])

readings = pd.DataFrame(np.vstack([normal, faulty]), columns=["temperature", "vibration"])
readings["truly_faulty"] = [0] * 480 + [1] * 20

sns.scatterplot(data=readings, x="temperature", y="vibration", hue="truly_faulty", palette=["tab
:blue", "tab:orange"])
plt.title("Motor readings (orange = faulty, but the model won't know)")
plt.show()
```

`np.random.RandomState(42)` creates a reproducible source of pseudo-random numbers. `rng.normal(50, 3, 480)` draws 480 values from a normal distribution with mean 50 and standard deviation three. Here they represent healthy motor temperatures in degrees Celsius.

The healthy vibration values have mean 2.0 and standard deviation 0.4, measured in millimetres per second. The 20 faulty observations have higher temperature and vibration means, with their own variation. The distributions overlap, so some healthy readings look unusual and some faulty readings look ordinary.

`column_stack` pairs temperatures and vibrations into two-column arrays. `vstack` places the faulty rows below the healthy rows. The table therefore has 500 rows. The list expression `[0] * 480 + [1] * 20` creates the known simulated status for later evaluation.

This simulation generates temperature and vibration separately within each status group. It does not reproduce every dependency or operating condition of a real motor. It provides a controlled example where we can count detection errors.

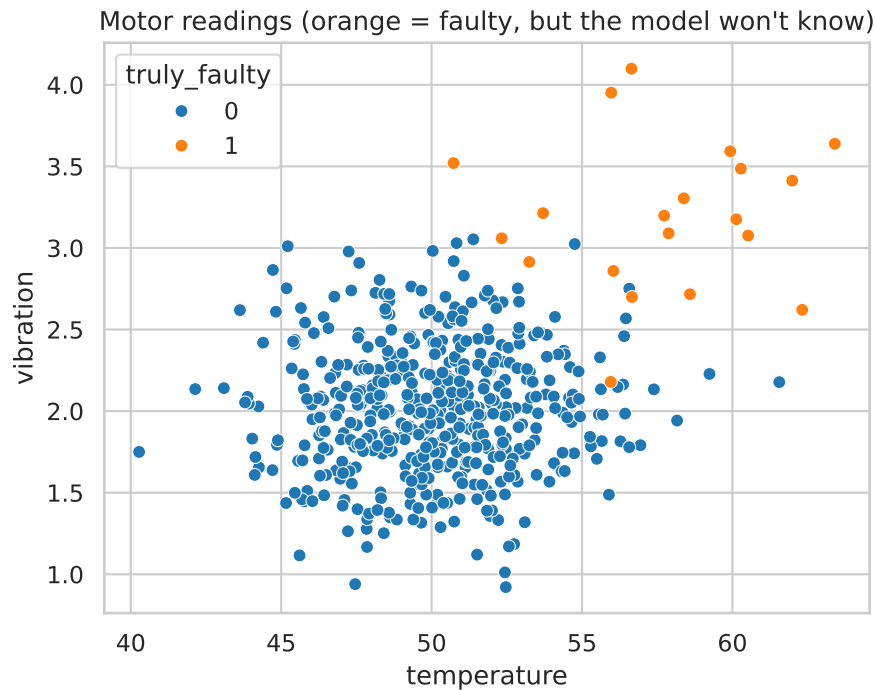


Figure 14: Simulated healthy and faulty readings. The colour uses the known simulation status.

12.2 How Isolation Forest identifies unusual points

Isolation Forest creates random partitions of the feature space. A partition chooses a feature and a split value. An isolated point can often be separated after relatively few cuts. A point within a dense region generally needs more cuts before it is isolated.

The method combines this behaviour across many random trees to form an anomaly score. It does not need a fault label for fitting. An unusual score indicates a point that is easy to isolate under this procedure. Whether the point represents a fault requires additional evidence.

```
from sklearn.ensemble import IsolationForest

iso = IsolationForest(contamination=0.04, random_state=42)  # expect about 4% odd readings
readings["flagged"] = (iso.fit_predict(readings[["temperature", "vibration"]]) == -1).astype(int)

caught = ((readings.flagged == 1) & (readings.truly_faulty == 1)).sum()
false_alarms = ((readings.flagged == 1) & (readings.truly_faulty == 0)).sum()
missed = ((readings.flagged == 0) & (readings.truly_faulty == 1)).sum()
print(f"Flagged: {readings.flagged.sum()} | faults caught: {caught}/20 | false alarms: {false_alarms} | missed: {missed}")
```

Only temperature and vibration are passed to the model. The column `truly_faulty` is excluded, preserving the unsupervised fitting task.

`contamination=0.04` sets an assumed outlier fraction for choosing a threshold on the fitted scores. Four percent of 500 is 20, so this run flags 20 observations. It does not tell the model which observations are faulty, and it does not establish the true fault rate in a future dataset.

`fit_predict` returns `-1` for predicted outliers and `1` for predicted inliers. Comparing with `-1` produces a Boolean array. `astype(int)` converts True to one and False to zero, making a convenient flag column.

The expressions for caught, false alarms, and missed compare two Boolean conditions element by element. The & symbol means both conditions must be true. Parentheses ensure each comparison is performed before combining them. Summing the True values counts the matching rows.

12.3 Understanding the detection result

The run flags 20 readings: 13 actual faults and seven healthy readings. Seven faults are missed. The table is

	Flagged	Not flagged
Actual faulty	13	7
Actual healthy	7	473

Fault recall is $13/20 = 65\%$. Alert precision is also $13/20 = 65\%$ here. Overall accuracy would be $(13 + 473)/500 = 97.2\%$. A system that called every reading healthy would already achieve $480/500 = 96\%$ accuracy while detecting no faults. This shows why accuracy alone gives an incomplete account of a rare-event detector.

```
sns.scatterplot(data=readings, x="temperature", y="vibration", hue="truly_faulty", palette=["tab
:blue", "tab:orange"], alpha=0.6)
flag = readings[readings.flagged == 1]
plt.scatter(flag.temperature, flag.vibration, facecolors="none", edgecolors="black", s=160,
linewidths=1.5, label="flagged")
plt.legend()
plt.title("Isolation Forest: circled readings were flagged")
plt.show()
```

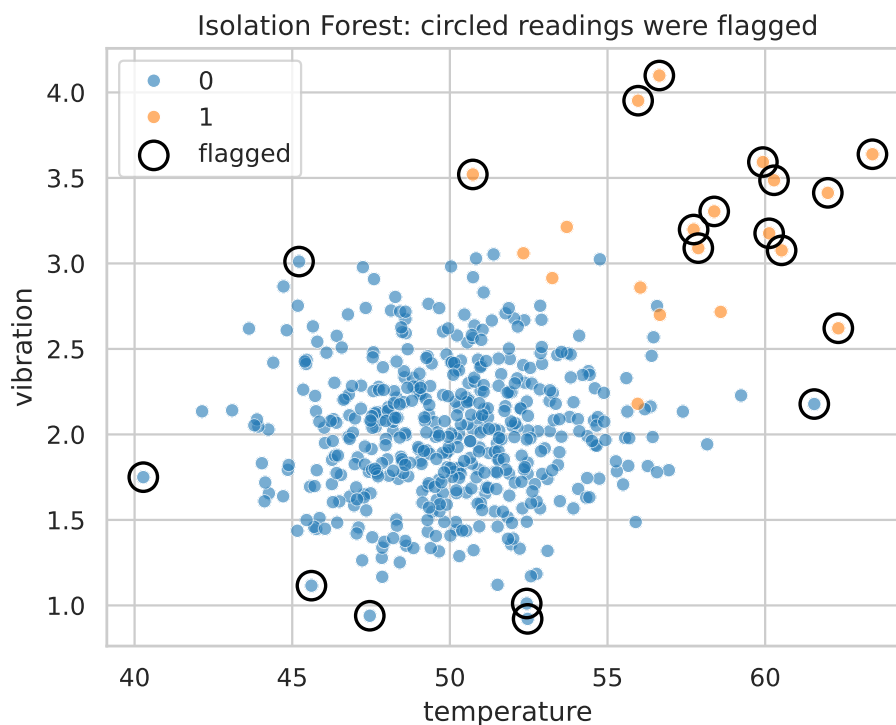


Figure 15: Black circles mark the readings flagged by Isolation Forest.

The second scatter call adds empty black circles around flagged points. `facecolors="none"` keeps

their original status colour visible. `readings[readings.flagged == 1]` selects the rows to circle. Many false alarms lie at the edge of the healthy cloud. Some missed faults overlap the healthy cloud. The method judges the geometry of the measurements, so overlapping distributions naturally make the task difficult.

These counts evaluate detection within the same simulated collection used to fit the unsupervised detector. They do not estimate performance on a later stream of unseen motor readings. That requires a separate evaluation representing the operating conditions of interest.

13 Notebook 2 exercises, with explanations

Material covered: Notebook 2 exercises and solutions; homework in slide 34.

13.1 Exercise 1: include age in customer clustering

```
# Exercise 1
X3 = StandardScaler().fit_transform(mall[["Age", "income", "spending"]])
sils = {k: silhouette_score(X3, KMeans(n_clusters=k, random_state=42, n_init=10).fit_predict(X3)
    ) for k in range(2, 11)}
best3 = max(sils, key=sils.get)
print("Best k with age included:", best3, f"(silhouette {sils[best3]:.3f})")
mall["cluster3"] = KMeans(n_clusters=best3, random_state=42, n_init=10).fit_predict(X3)
print(mall.groupby("cluster3")[["Age", "income", "spending"]].mean().round(1))
```

The new input has three columns: age, income, and spending. Standardisation centres each column and divides by its standard deviation so that their units do not directly determine the distance weights. This is an exploratory clustering fit on the current customer collection.

The dictionary comprehension fits a clustering for each k from two through ten and stores its silhouette score. `max(sils, key=sils.get)` returns the dictionary key whose stored score is largest. The best tested value becomes six, with silhouette about 0.428.

The six mean profiles are:

Group	Age	Income	Spending
0	56.3	54.3	49.1
1	26.8	57.1	48.1
2	41.9	88.9	17.0
3	32.7	86.5	82.1
4	25.0	25.3	77.6
5	45.5	26.3	19.4

The broad middle-income, middle-spending group divides into older and younger groups. This is understandable because age now participates directly in the distance calculation. It previously appeared only in the descriptive summary.

The silhouette of 0.428 should not be treated as a direct failure relative to 0.554. The feature space and its scaling have changed, so the scores describe different geometries. Decide whether the additional age distinction is useful for the question you are studying.

13.2 Exercise 2: change the alert threshold

```
# Exercise 2
for c in [0.02, 0.04, 0.08]:
    f = IsolationForest(contamination=c, random_state=42).fit_predict(readings[["temperature", "vibration"]]) == -1
    t = readings.truly_faulty == 1
    print(f"contamination {c}: caught {(f & t).sum()}/20, false alarms {(f & ~t).sum()}")
```

The variable `f` is a Boolean array of predicted flags. The variable `t` is a Boolean array of true simulated faults. `f & t` selects detected faults; `f & ~t` selects false alarms, where `~` reverses the Boolean condition.

Contamination	Flagged	Faults caught	False alarms	Faults missed
0.02	10	9	1	11
0.04	20	13	7	7
0.08	40	16	24	4

Raising the assumed outlier fraction moves the score threshold so more readings become alerts. The underlying random partitions are controlled by the same seed here. At 0.02, precision is $9/10 = 90\%$ and recall is $9/20 = 45\%$. At 0.08, precision is $16/40 = 40\%$ and recall is $16/20 = 80\%$.

The speaker's oxygen-plant example asks you to consider error costs. If missed faults have a high cost, a more sensitive threshold may be worth investigating. False alarms also consume inspection capacity and can create alarm fatigue. Choose an operating threshold using representative validation evidence and the actual consequences of each error, rather than treating a particular contamination value as a safety guarantee.

13.3 Exercise 3: breast cancer measurements in two dimensions

```
# Exercise 3
from sklearn.datasets import load_breast_cancer
bc = load_breast_cancer()
bc2 = PCA(n_components=2).fit(StandardScaler().fit_transform(bc.data))
pts = bc2.transform(StandardScaler().fit_transform(bc.data))
sns.scatterplot(x=pts[:, 0], y=pts[:, 1], hue=pd.Series(bc.target).map({0: "malignant", 1: "benign"}), alpha=0.7)
plt.title(f"Breast cancer in 2D (keeps {bc2.explained_variance_ratio_.sum():.0%} of the information)")
plt.show()
```

This code standardises all 30 measurements before exploratory PCA because their ranges differ. The first two components retain about 63% of the variance in that standardised dataset. The plot colours examples using diagnosis for interpretation.

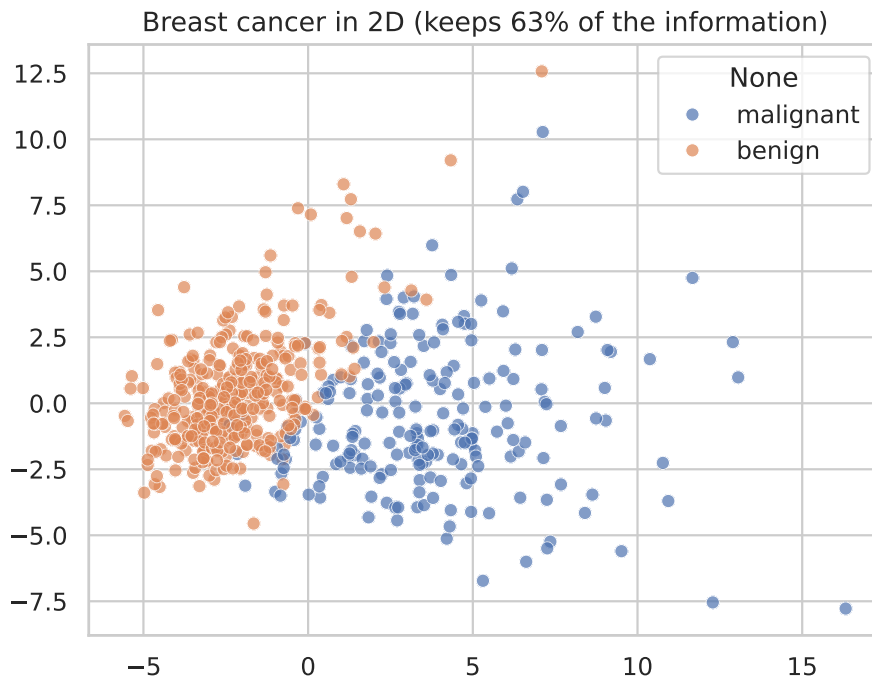


Figure 16: Two-component PCA view of the standardised breast cancer measurements.

The supplied solution fits `StandardScaler` twice on the same full data. Both fitted transformations are the same here, so the coordinates remain consistent, but the repeated fit is unnecessary. A clearer exploratory version is:

```
scaled_bc = StandardScaler().fit_transform(bc.data)
bc_pca = PCA(n_components=2)
points = bc_pca.fit_transform(scaled_bc)
```

This plot describes the available dataset. For an evaluated predictive model, fit the scaler and PCA inside its training pipeline. The amount of retained variance says nothing by itself about how many diagnoses a classifier would get right.

14 Notebook 3: learning when only a few labels are available

Material covered: Slides 28–29 and 32; Notebook 3, Parts 1–2.

14.1 Features can be available before their answers

A camera can collect images quickly. Assigning a trustworthy category to each image may require a person’s time. Sensor measurements can accumulate automatically while fault labels require inspection. Semi-supervised learning uses a training collection that contains some labelled examples and some unlabelled examples.

An unlabelled digit image still contains its 64 pixel values. What is missing is the confirmed answer, such as “this image is a seven”. Its position relative to other images can nevertheless help a learning procedure.

This notebook creates a controlled experiment by hiding labels from a fully labelled teaching dataset. The true hidden labels remain available to the analyst for later checking. The fitting code receives

the deliberately incomplete target array.

14.2 Start by separating the test images

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.semi_supervised import SelfTrainingClassifier, LabelSpreading

digits = load_digits()
X_train, X_test, y_train, y_test = train_test_split(
    digits.data, digits.target, test_size=0.2, random_state=42, stratify=digits.target)
print("Training images:", len(X_train), "| Test images:", len(X_test))
```

The setup imports the tools again so Notebook 3 can run independently. It loads the same digit dataset and makes a stratified 80/20 split with seed 42. There are 1,437 training images and 360 test images.

The test images and their labels are reserved for evaluation in this example. They are not added to the graph or self-training pool. The semi-supervised methods use only the 1,437 training images, with most of their answers hidden.

14.3 Hiding all but 50 labels

```
N_LABELS = 50
rng = np.random.RandomState(0)
labelled_idx = rng.choice(len(y_train), N_LABELS, replace=False)

y_partial = np.full(len(y_train), -1) # everything unlabelled...
y_partial[labelled_idx] = y_train[labelled_idx] # ...except 50 images

print(f"Labelled: {(y_partial != -1).sum()} | Unlabelled: {(y_partial == -1).sum()}")
print("Labels per digit:", np.bincount(y_train[labelled_idx]))
```

`N_LABELS=50` sets the label budget. `rng.choice` chooses 50 positions from the 1,437 training positions. `replace=False` ensures that each selected position is distinct. The seed zero controls this selection independently of the earlier train/test split.

`np.full(len(y_train), -1)` creates an array with 1,437 entries, all initially `-1`. In these semi-supervised estimators, `-1` means the target is unlabelled. It is not a digit category. The next line copies the true labels only at the selected positions.

The input features remain available for every training image. Only the target array changes. The resulting counts are 50 labelled images and 1,387 unlabelled images:

$$\frac{50}{1437} \times 100 \approx 3.48\% \quad \text{labelled.}$$

`np.bincount` counts the selected labels for digits zero through nine. It produces

[6, 5, 5, 4, 6, 5, 3, 6, 6, 4].

There are three labelled sixes and six labelled zeros, for example. “About five per digit” describes the average. The random selection does not force exactly five examples per class. With a smaller budget or different seed, some classes may be absent, which can make learning much harder.

14.4 The baseline using the 50 confirmed answers

```
def make_model():
    return make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))

baseline = make_model().fit(X_train[labelled_idx], y_train[labelled_idx])
acc_baseline = baseline.score(X_test, y_test)
print(f"Only 50 labels: {acc_baseline:.1%}")
```

`make_model` is a function that returns a new scaler-plus-logistic-regression pipeline each time it is called. This prevents one experiment from accidentally continuing with another experiment’s fitted model.

The baseline receives the 50 selected feature rows and their 50 confirmed labels. Both its scaler and classifier are fitted using those rows. It is then evaluated on all 360 test images.

The baseline achieves 86.1% accuracy, corresponding to 310 correct predictions out of 360. A small labelled subset can already carry useful information. Its performance also depends on which images were selected and how well they represent the classes.

This is the reference for measuring what the unlabelled training images contribute under the methods that follow. The fully labelled model appears later as a separate reference.

15 Self-training: learning from selected predictions

Material covered: Slide 30; Notebook 3, Part 3.

15.1 Following one self-training round

Self-training starts with a supervised classifier trained on the confirmed labels. It then predicts probabilities for unlabelled examples. Predictions that pass a confidence threshold are accepted as **pseudo-labels**. The classifier is trained again using the original labelled examples and the newly accepted examples. This can repeat for several rounds.

Consider an unlabelled image for which the model assigns probability 0.94 to digit seven and distributes the remaining probability among other digits. With threshold 0.9, that prediction can be accepted as a pseudo-label. If another image’s highest probability is 0.62, it remains unlabelled in that round.

A pseudo-label is generated by the model. Its correctness has not been confirmed by a person. A confident error can enter the next training round and influence later predictions, so the threshold affects the balance between additional data and additional mistakes.

15.2 The code and learned attributes

```
self_train = SelfTrainingClassifier(make_model(), threshold=0.9)
self_train.fit(X_train, y_partial)

added = (self_train.labeled_iter_ > 0).sum()
acc_self = self_train.score(X_test, y_test)
print(f"Self-training added {added} images as new labels")
print(f"Accuracy: {acc_self:.1%}")
```

`SelfTrainingClassifier` wraps the fresh pipeline returned by `make_model()`. The underlying estimator must provide probabilities through `predict_proba`. The fit receives all training features and the partially labelled target array.

The implementation accepts a pseudo-label when its highest predicted probability is above the threshold. The notebook’s “at least 90%” wording gives the intended intuition; the implemented comparison is strictly greater than 0.9. Default stopping conditions include adding no new labels, labelling all examples, or reaching the iteration limit, which is ten in the checked version.

`labeled_iter_` records when each training example received a label. Zero identifies original labelled examples, positive values identify accepted pseudo-labels from later rounds, and `-1` indicates an example still unlabelled. Comparing with zero and summing counts only the new additions.

This run accepts 1,172 pseudo-labels. Together with the original 50 labels, that gives 1,222 labelled or pseudo-labelled training examples and leaves 215 without an accepted label. Test accuracy is 90.0%, or 324 correct predictions out of 360.

The increase from 86.1% to 90.0% is about 3.9 percentage points, representing 14 additional correct predictions on this test collection. The code does not establish that every accepted pseudo-label is correct.

15.3 Why confidence needs interpretation

A probability of 0.9 is the model’s estimate. A **well-calibrated** model would have roughly 90% correct predictions among a large collection of comparable predictions given confidence near 0.9. Training on only 50 examples can make that calibration uncertain.

Raising the threshold can admit fewer mistakes, but may leave too few new examples to improve the model. Lowering it can add useful examples and incorrect labels together. The exercise later measures this effect for three thresholds.

In this particular pipeline, each fitting round also refits the scaler on the currently accepted training set. As more pseudo-labelled examples are admitted, the scaler’s fitted statistics may change. Placing the preprocessing inside the wrapped model keeps that transformation linked to the classifier used in the same round.

16 Label spreading: using the geometry of all training images

Material covered: Slides 30–31; Notebook 3, Part 4.

16.1 A graph of similar examples

A graph consists of nodes and connections. Here each node represents one training image. Connections express similarity in its pixel features. The KNN graph setting uses a neighbourhood size of seven to build local relationships among the images.

Imagine a sequence of similar handwritten twos. One image has a confirmed label. Its nearby images resemble it, and their nearby images continue along the same pattern. The unlabelled images help describe that structure, allowing label information to influence a wider group.

The key assumption is that connected, similar examples often share labels. A graph that connects different classes strongly can spread incorrect information. A graph broken into small disconnected pieces can leave parts with little useful labelled support.

Label spreading maintains a distribution over possible classes at each node. Its updates combine information from the graph with the initial label information. The default `alpha=0.2` controls that mixing through soft clamping. The algorithm's numerical details are more involved than passing a single fixed label to every neighbour, but the local-similarity idea explains its purpose.

16.2 Fitting and evaluating label spreading

```
spread = LabelSpreading(kernel="knn", n_neighbors=7)
spread.fit(X_train, y_partial)
acc_spread = spread.score(X_test, y_test)

hidden = y_partial == -1
guess_acc = (spread.transduction_[hidden] == y_train[hidden]).mean()
print(f"Label spreading accuracy on test images: {acc_spread:.1%}")
print(f"Its guesses for the 1,387 hidden labels were right {guess_acc:.1%} of the time")
```

`kernel="knn"` selects the nearest-neighbour graph option. `n_neighbors=7` controls the neighbourhood size. The fit uses all 1,437 training images and the incomplete labels.

This notebook supplies raw pixel intensities to label spreading. They share the same intensity units. The baseline and self-training logistic models use a scaler, so the comparison includes differences in both model family and preprocessing. Scaling the graph inputs would define another experiment.

`spread.score(X_test, y_test)` predicts labels for the separate test images and measures accuracy. `transduction_` contains the inferred labels of the training images used during the graph fit. The Boolean mask `hidden` selects positions whose original target was `-1`.

Comparing those inferred training labels with `y_train[hidden]` checks the hidden answers, which are available only because this is a simulated label-hiding exercise. The comparison happens after fitting. In a real unlabelled collection, those true answers would require additional labelling before this check could be made.

16.3 The supplied results and the verification run

The teaching materials report 94.7% test accuracy and 90.8% accuracy on hidden training labels. Running the supplied code in the environment used for these notes produced 95.3% test accuracy

and 91.1% hidden-label accuracy.

The 95.3% test score corresponds to 343 correct predictions out of 360. The reported slide score of 94.7% corresponds to 341 correct predictions. That is a difference of two test images. The original execution environment was not supplied in enough detail to identify the exact cause. Neighbour ties, numerical libraries, and implementation versions can affect graph-based results. The difference should be recorded without claiming a proven cause.

Both runs show a substantial improvement over the 50-label baseline in this example. The test score and hidden-training-label score measure different things. One evaluates new images; the other evaluates inferred labels within the training graph. Their denominators and roles differ.

16.4 Inspecting individual inferred labels

```
examples = np.where(hidden)[0][:12]
fig, axes = plt.subplots(2, 6, figsize=(11, 4))
for ax, i in zip(axes.ravel(), examples):
    ax.imshow(X_train[i].reshape(8, 8), cmap="gray_r")
    guess, truth = spread.transduction_[i], y_train[i]
    ax.set_title(f"guess {guess}" + (" (" if guess == truth else f" (true {truth})"),
                color="black" if guess == truth else "red")
    ax.axis("off")
plt.suptitle("Labels guessed by label spreading")
plt.show()
```

`np.where(hidden)[0]` obtains the training positions whose labels were hidden. `[:12]` selects the first twelve of those positions. This is an illustration, not a random sample of errors or a full quality assessment.

`plt.subplots(2, 6)` creates twelve axes. `axes.ravel()` flattens their two-dimensional arrangement into a sequence for the loop. `reshape(8, 8)` restores a flattened 64-value row to its image layout.

The title shows the inferred digit. If it differs from the known hidden answer, the title also includes the true digit and is coloured red. This makes specific mistakes easier to discuss.

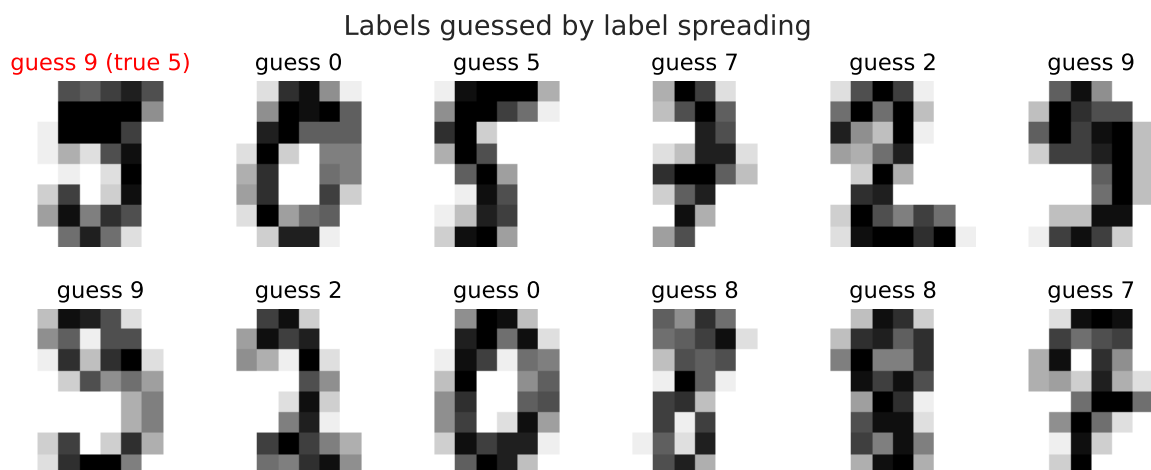


Figure 17: Twelve hidden training labels inferred by label spreading in the verification run.

A visual review can reveal ambiguous handwriting or systematic confusions. Checking only a few attractive examples cannot establish overall reliability. Combine individual inspection with

representative quantitative evaluation.

16.5 Comparing the four approaches

```
full = make_model().fit(X_train, y_train)
acc_full = full.score(X_test, y_test)

summary = pd.Series({
    "Only 50 labels": acc_baseline,
    "Self-training": acc_self,
    "Label spreading": acc_spread,
    "All 1,437 labels": acc_full,
})
summary.plot(kind="barh", color=["grey", "tab:blue", "tab:orange", "black"], xlim=(0.8, 1.0))
plt.xlabel("test accuracy")
plt.title("50 labels + 1,387 unlabelled images")
plt.show()
summary.round(3)
```

The fully labelled reference uses `make_model()` with all 1,437 training labels. It reaches 97.2%, or 350 correct predictions out of 360. A Pandas Series stores the four scores with readable names and plots a horizontal bar chart.

Approach	Supplied material	Verification run
Only 50 labels	86.1%	86.1%
Self-training	90.0%	90.0%
Label spreading	94.7%	95.3%
All 1,437 labels, logistic regression	97.2%	97.2%

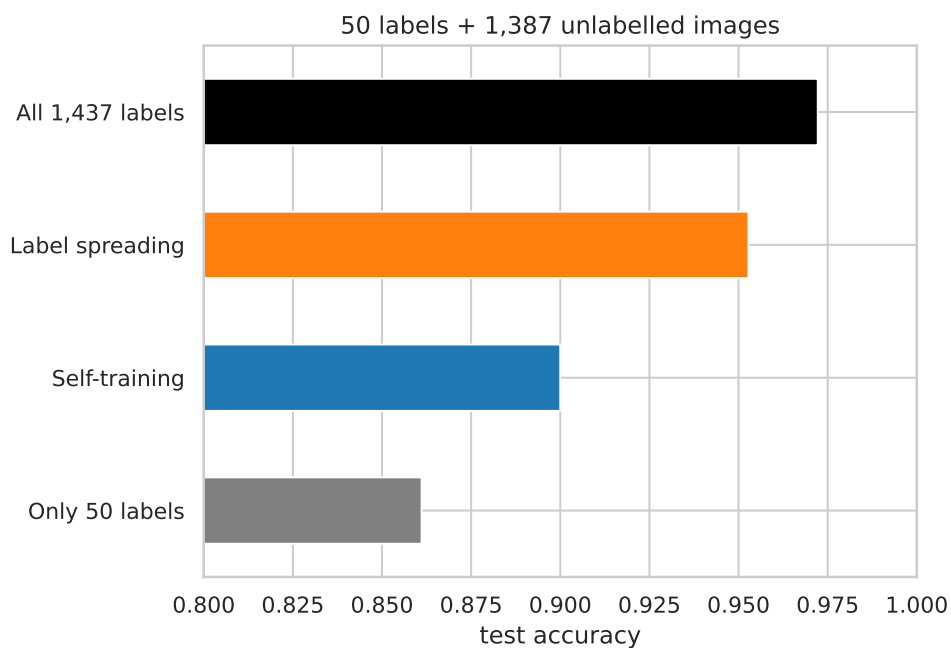


Figure 18: The four test accuracies from the verification run. The horizontal axis begins at 0.8.

The supplied label-spreading improvement is 8.6 percentage points over the baseline, with a gap of

2.5 points from the fully labelled reference. In the verification run, these become about 9.2 and 1.9 points. Always specify whether a change is a difference in percentage points or a relative percentage change.

The chart's `xlim=(0.8, 1.0)` magnifies the visible differences by showing only the upper portion of the accuracy scale. Read the numerical values alongside the bars.

The fully labelled logistic model is a reference model, not a maximum achievable accuracy for the dataset. A different model can outperform it even with fewer labels. Also, unlabelled data still has collection, storage, cleaning, and processing costs; the saving here concerns the withheld manual labels.

17 How many labels do we need?

Material covered: Notebook 3, Part 5; speaker notes for slide 32.

17.1 The label-budget experiment

```
rows = []
for n in [20, 50, 100, 200]:
    idx = np.random.RandomState(0).choice(len(y_train), n, replace=False)
    yp = np.full(len(y_train), -1); yp[idx] = y_train[idx]
    only = make_model().fit(X_train[idx], y_train[idx]).score(X_test, y_test)
    ls = LabelSpreading(kernel="knn", n_neighbors=7).fit(X_train, yp).score(X_test, y_test)
    rows.append({"labels": n, "only labelled": only, "label spreading": ls})

curve = pd.DataFrame(rows).set_index("labels")
curve.plot(marker="o")
plt.axhline(acc_full, color="black", linestyle="--", label="all 1,437 labels")
plt.ylabel("test accuracy"); plt.legend()
plt.title("Accuracy vs number of labels")
plt.show()
curve.round(3)
```

The loop tries 20, 50, 100, and 200 confirmed labels. For each budget, it chooses positions using a newly created random generator with seed zero, constructs a partial target array, fits a baseline on only the selected labelled rows, and fits label spreading on all training features.

Reusing the seed makes the experiment reproducible, and these selections in the checked run form progressively larger subsets of the same random ordering. The experiment still represents one particular selection process. Repeating across several seeds would show how sensitive the scores are to which examples receive labels.

The dictionaries collected in `rows` become a table indexed by label count. The plot draws one curve for the labelled-only baseline and one for label spreading. The dashed line marks the fully labelled logistic reference.

Labels	Labelled-only	Spreading: supplied	Spreading: verified
20	68.6%	80.8%	81.1%
50	86.1%	94.7%	95.3%
100	87.2%	96.4%	96.4%
200	91.1%	98.1%	98.1%

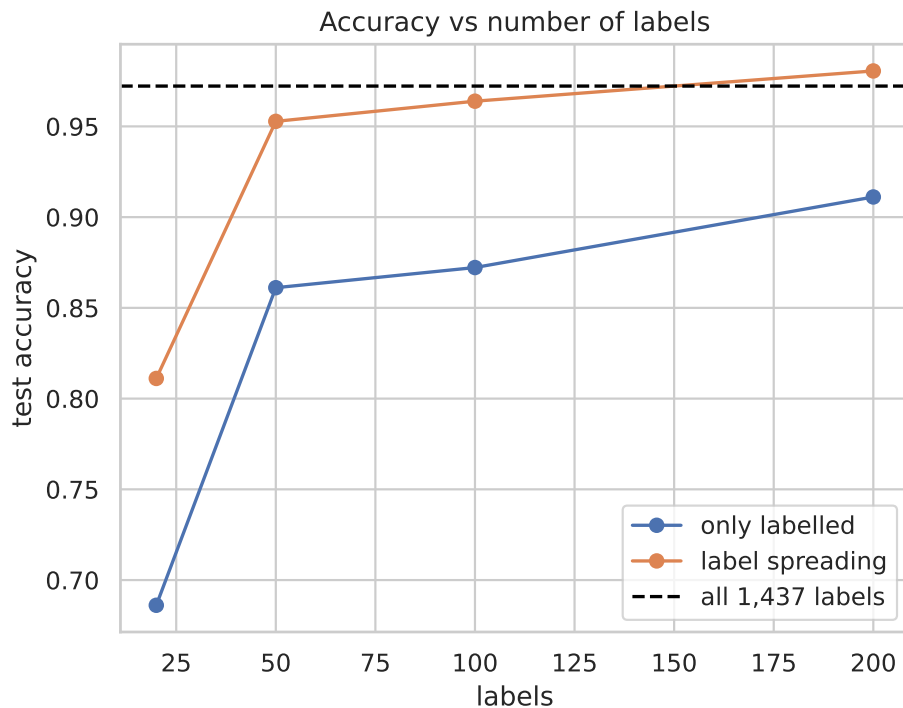


Figure 19: Accuracy changes with the confirmed-label budget in the verification run.

Twenty labels leave very little confirmed information for ten classes. With 50, the graph receives enough useful anchors to improve substantially in this example. With 200, label spreading obtains 98.1%, exceeding the 97.2% fully labelled logistic reference on this test collection.

That result compares different model families. It does not show that withholding labels intrinsically improves learning. The graph's assumptions happen to work well on this digit representation. Another fully supervised model could produce a different comparison.

The notebook uses the test set repeatedly to illustrate budgets and settings. If those experiments choose the final method, treat these scores as development evidence and use a further independent evaluation for a final claim. With scarce labels, reserve evaluation labels deliberately rather than allowing them to enter the pseudo-labelling process.

17.2 Exercise 1: change the graph neighbourhood

```
# Exercise 1
for k in [3, 7, 15]:
    print(f"n_neighbors={k:2d}: {LabelSpreading(kernel='knn', n_neighbors=k).fit(X_train,
        y_partial).score(X_test, y_test):.1%}")
```

The results are:

Neighbours	Supplied accuracy	Verified accuracy
3	57.8%	57.8%
7	94.7%	95.3%
15	94.7%	94.7%

With three neighbours, the graph can have poorly connected regions that receive little useful label

information. This is a plausible explanation for the sharp reduction and the explanation offered by the speaker notes; inspecting graph connectivity would be needed to confirm the detailed cause.

With more connections, labels can reach a broader set of related images. Too many connections can also link different classes and spread errors. Seven is the highest observed score in this verification run; seven and fifteen tie in the supplied results. Select the setting using an appropriate validation procedure.

17.3 Exercise 2: change the self-training threshold

```
# Exercise 2
for th in [0.7, 0.9, 0.99]:
    st = SelfTrainingClassifier(make_model(), threshold=th).fit(X_train, y_partial)
    print(f"threshold {th}: added {(st.labeled_iter_ > 0).sum():4d} images, accuracy {st.score(
        X_test, y_test):.1%}")
```

Threshold	Pseudo-labels added	Test accuracy
0.70	1,318	88.9%
0.90	1,172	90.0%
0.99	16	86.7%

These results reproduce the supplied materials. At 0.70, many predictions are accepted, including less confident ones. Accuracy falls below the 0.90 run even though the accepted set is larger. At 0.99, only sixteen examples are added, so the procedure changes relatively little from the baseline.

The experiment illustrates how both the quantity and quality of pseudo-labels matter. It does not establish 0.90 as an optimal threshold for every dataset. Calibration, class coverage, label noise, and the similarity between labelled and unlabelled examples all affect the result.

17.4 Exercise 3: why messy data can make spreading fail

Suppose a blurry three lies near several eights in pixel space. The graph can connect it to those eights and reinforce the wrong label. Backgrounds, lighting, sensor noise, or irrelevant features can make distance reflect properties that are poorly aligned with the desired category.

The method can also struggle when the labelled examples omit a class or cover only a narrow part of it. A cluster of new handwriting styles may have little connection to trustworthy anchors. An unlabelled collection drawn from a different source may introduce structures that do not match the task.

Useful checks include inspecting examples from several graph regions, measuring performance on separately labelled validation data, checking class coverage, and testing sensitivity to representation and neighbourhood size. Hidden answers from the simulated exercise must remain excluded from fitting and setting selection if they are intended for evaluation.

18 Bringing the lesson together

Material covered: Slides 33–35; speaker questions and glossary.

18.1 Choosing an initial method

The slide’s model guide gives starting points for experiments. Begin by defining the target, available inputs, evaluation measure, and intended use. Then compare suitable candidates with a consistent evaluation procedure.

Task	Starting methods in the lesson	What to examine
Predict a number	Linear regression, random forest	Error in the target’s units, outliers, nonlinear patterns.
Predict a category with a readable rule	Logistic regression, decision tree	Class-specific errors, coefficients or paths, complexity.
Prediction from a table	Gradient boosting, random forest	CV results, interactions, missing-value handling.
Classification on a manageable dataset	SVM	Scaling, C , kernel settings, fitting cost.
Explore groups without labels	K-Means, hierarchical clustering	Feature choices, distance, stability, usefulness.
Create fewer input coordinates	PCA	Scaling, variance retained, downstream accuracy.
Flag unusual observations	Isolation Forest	Alert precision, missed events, threshold.
Use a small labelled subset	Self-training, label spreading	Label coverage, confidence, graph quality.
Learn from images, sound, or text	Neural networks, introduced on Day 4	Representation, data size, computation, validation.

A small dataset does not automatically favour SVM, and a table does not automatically favour boosting. The experiments today demonstrate that the model ranking depends on the data representation and tested settings. Images can also support simpler baselines, as the logistic digit classifier shows.

18.2 Answers to the classroom discussion questions

Why can logistic regression perform so well here? The breast cancer features already summarise useful properties of cell nuclei. A weighted boundary can capture much of their relationship with diagnosis. The observed CV score supports that model for this experiment. The complexity of a method alone cannot determine its usefulness.

Why might an understandable model be valuable? A simpler decision rule can make it easier to inspect which measurements influence a prediction, identify input problems, and communicate assumptions. Interpretability still depends on preprocessing, correlated inputs, and the audience’s needs. High stakes also require appropriate evidence about performance.

How many folds should I use? Five folds are a reasonable teaching choice here. The dataset size, rarest class count, dependence between observations, and computation budget affect the decision. A stratified split needs enough examples of each class to populate the desired folds. Repeated or nested validation can answer different questions, but each has a computational cost.

How does a pipeline prevent the age-imputation leak? Each CV fit learns its own median from its training rows. The same median is then used to transform that fit's validation rows. The implementation follows the data separation required by the evaluation.

Do the customer groups have a correct answer? There are no supplied target customer categories in this exercise. The groups are evaluated through their geometry, stability under reasonable changes, and relevance to the intended question. A useful segmentation may still need human interpretation and further testing.

How is self-supervised learning related to this lesson? In self-supervised learning, training targets are constructed from the data itself, such as predicting a hidden part of an input. Semi-supervised learning, as practised here, combines a confirmed labelled subset with additional unlabelled examples. A larger learning system can use both kinds of training at different stages. Language-model pretraining is therefore not the same procedure as today's digit label-spreading experiment.

Can semi-supervised learning make performance worse? Yes. Incorrect pseudo-labels or graph connections can move learning in an unhelpful direction. The threshold exercise already shows that accepting more examples can lower accuracy. Keep an independently labelled evaluation set to assess the effect.

18.3 Before moving to Day 4

Finish the three notebooks and explain each main result aloud. For the random-forest homework, state the grid, fold count, winning settings, and comparison score. For customer clustering with age, describe how the feature representation changes the distances and why the broad central group separates by age.

Day 4 introduces neural networks, deep learning, and computer vision. The speaker notes ask students to use a GPU runtime for that session. The evaluation principles from today continue to apply: consistent inputs, training-only fitting of learned preprocessing, suitable validation, and a final assessment that has not guided development.

19 A short self-check, with worked answers

19.1 Questions

1. A tree scores 100% on training data and 75% on validation data. What should you investigate?
2. In today's breast cancer CV, how many rows are used for training and validation in each fold?
3. What does a fold spread of one percentage point tell you?
4. Why should a scaler be inside a CV pipeline?
5. A search has three depth values, two tree counts, and five folds. How many CV fits does it run?
6. A customer lies at (4,6) and a cluster centre at (1,2). What is the Euclidean distance?
7. A point has average within-cluster distance two and nearest-other-cluster average distance five. What is its silhouette?
8. Why can two PCA coordinates retain substantial variance while giving poor digit classification?
9. A detector flags 30 readings, including 18 of 24 actual faults. What are precision and recall?
10. In the semi-supervised notebook, does -1 mean that an image is an unusual digit?

11. Why is a model confidence of 0.99 insufficient to prove a pseudo-label correct?
12. Why can a 200-label graph model beat a fully labelled logistic model in one comparison?

19.2 Worked answers

1. The 25-point gap suggests poor generalisation. Check model flexibility, split quality, consistent preprocessing, and possible distribution changes. A shallower tree is one candidate to evaluate using the development data.
2. There are 455 development rows. Five equal folds contain 91 each. Every run trains on $4 \times 91 = 364$ and validates on 91.
3. It describes the standard deviation of the five observed fold scores. It does not define a guaranteed range for future accuracy.
4. The scaler must learn its statistics from each fold's training rows. Putting it inside the pipeline makes those statistics part of the fit being evaluated.
5. There are $3 \times 2 = 6$ settings and $6 \times 5 = 30$ CV fits. Default refitting adds one further fit of the selected setting.
6. The distance is $\sqrt{(4-1)^2 + (6-2)^2} = \sqrt{9+16} = 5$.
7. The silhouette is $(5-2)/5 = 0.6$. The point is more compactly associated with its own cluster under this distance calculation.
8. PCA maximises retained input variance. Some smaller-variance directions can carry distinctions between digit labels, and different images can overlap after projection.
9. Precision is $18/30 = 60\%$. Recall is $18/24 = 75\%$. There are 12 false alarms and six missed faults.
10. It marks an unavailable training target. The image's pixel values remain ordinary input features.
11. Confidence is an estimated probability from the fitted model. Miscalibration or unfamiliar inputs can produce confident errors.
12. The model families and their representations differ. The graph can fit the digit structure well. The fully labelled logistic model is one reference, so its score is not an upper bound.

20 Practical troubleshooting

Symptom	What to check
<code>NameError</code> for <code>cv</code> or <code>X_train</code>	Run the earlier setup, split, and CV cells in the same notebook. A restarted runtime forgets previously created variables.
A download fails	Check the runtime's connection and the URL. The mall CSV and Titanic loader can require network access. Built-in breast cancer and digit loaders do not download these teaching datasets.
Grid search appears slow	It fits many models. The SVM grid runs 80 CV fits plus a refit. Wait for the active cell to finish before running another copy.
A convergence warning appears	Read the warning with warning suppression removed. Check scaling and the relevant iteration limit. A warning is evidence to investigate even when a score is printed.

Symptom	What to check
Cluster numbers differ	Compare centroids, counts, and profiles. The numerical identifiers can be permuted while the groups remain the same.
Scores differ slightly	Record package versions, random seeds, preprocessing, split rules, and full settings. Graph-neighbour ties and numerical differences can change a small number of predictions.
An estimator says it is unfitted	<code>cross_val_score</code> fits copies. Fit the original estimator on the intended training data before calling its prediction method directly.
A pipeline rejects new rows	Check required column names and types. The Titanic pipeline also expects the precomputed family-size column.
XGBoost cannot be imported	Follow the optional installation instruction if you want that exercise. The core scikit-learn comparisons can run without it.
Graph predictions are poor with few labels	Check which classes have confirmed examples, the feature representation, and neighbourhood settings. Keep evaluation labels outside fitting.
A PCA plot is mirrored	Component directions may have opposite signs across valid fits. Inspect relative geometry and explained variance before diagnosing a problem.

21 Glossary for quick revision

Term	Meaning in today's lesson
Generalisation	Performance on new examples from the intended task and population.
Underfitting	A fitted relationship that misses useful structure even in the training data.
Overfitting	Adapting too closely to training-specific detail, with weaker performance on new examples.
Bias and variance	Systematic error from assumptions; sensitivity of fitted predictions to changes in the training sample.
Validation data	Examples used to compare models or make development choices.
Final test data	Reserved examples used after development decisions have been fixed.
Fold	One subset used in a cross-validation partition.
Stratification	Preserving approximate class proportions when creating subsets.
Regularisation	Restrictions or penalties that control how a model fits the training examples.
Ensemble	A predictor that combines multiple fitted models.
Bootstrap sample	A sample of rows drawn with replacement.
Bagging	Combining models fitted to resampled versions of the training data.
Boosting	Sequentially adding models using the current ensemble's remaining loss information.
Support vector	A training example with a nonzero contribution to the fitted SVM decision function.
Kernel	A function computing similarity associated with a feature-space representation.
Hyperparameter	A setting chosen for the learning procedure, such as tree depth or neighbour count.

Term	Meaning in today's lesson
Pipeline	A sequence of fitted transformations followed, here, by a final estimator.
Leakage	Information entering development or fitting from a source that should be unavailable under the evaluation design.
Centroid	The coordinate-wise mean location of the examples in a cluster.
Inertia	The sum of squared distances from points to their assigned K-Means centres.
Silhouette	A score comparing within-cluster and nearest-other-cluster distances.
Ward linkage	A merge rule based on increases in within-group squared variation.
Dendrogram	A tree drawing of successive hierarchical merges.
Adjusted Rand index	A chance-adjusted comparison of two group assignments based on pairs of examples.
Principal component	A learned direction used to form a new coordinate in PCA.
Explained variance	Input variation accounted for by selected PCA components.
Anomaly	An observation considered unusual under the chosen model and feature representation.
Contamination	The outlier proportion used to set Isolation Forest's fitted score threshold in this example.
Pseudo-label	A model-generated target accepted for additional training.
Calibration	Agreement between predicted probabilities and observed frequencies over suitable groups of examples.
Transduction	Inference for the unlabelled examples included in the training collection.
Semi-supervised learning	Learning with confirmed labels for part of the training collection and unlabelled examples for the rest.

22 Coverage, verification, and further reading

22.1 Where the supplied material appears

All substantive code cells from the three supplied notebooks are included in these notes. Markdown prompts, classroom questions, exercises, and speaker explanations are developed in the corresponding sections.

Slides	Topic	Notebook connection
1–3	Introduction and recap	Shared setup and learning goals.
4–9	Overfitting and fair evaluation	Notebook 1, Parts 1–2.
10–15	Ensembles, SVM, comparison	Notebook 1, Part 3.
16–18	Tuning and preprocessing	Notebook 1, Parts 4–5 and exercises.
19–22	Unlabelled data and K-Means	Notebook 2, Parts 1–2.
23	Hierarchical clustering	Notebook 2, Part 3.
24–25	PCA	Notebook 2, Part 4.
26–27	Anomalies and practice	Notebook 2, Part 5 and exercises.
28–32	Semi-supervised learning	Notebook 3, all parts and exercises.
33–35	Choosing methods and recap	Closing discussion, self-check, and glossary.

Slide 32 compresses the notebook’s part labels in its hands-on summary. The notebook itself has five parts: hide labels, baseline, self-training, label spreading, and label budgets. These notes follow that complete notebook order.

22.2 What was checked

The three supplied notebooks were run in order with their original modelling choices and seeds. Their core model comparisons, clustering summaries, PCA results, motor counts, and exercise results were reproduced. Figures in these notes were generated from those executions.

Label spreading produced the small differences documented in its sections. The optional XGBoost package was absent, so its reported comparison is explicitly identified as a source-material result. No score from that optional classifier is claimed as independently reproduced.

The notes also clarify several teaching shortcuts: repeated use of the purported final test set; the distinction between variance and task-relevant information in PCA; the different preprocessing in the raw-PCA plot and scaled classification experiment; the arbitrary nature of cluster identifiers; and the limits of probability and anomaly interpretations.

22.3 Sources and documentation

The principal sources are the supplied Day 3 slide deck, Day 3 speaker notes, and the three notebooks. The explanations, worked arithmetic, and output interpretation follow those materials. The following primary documentation was consulted to check technical definitions and implementation details. Links to the stable documentation can change as scikit-learn is updated; the execution version for these notes was 1.8.0.

1. **Dataset definitions.** Scikit-learn, toy datasets, including breast cancer feature summaries and

digit data.

https://scikit-learn.org/stable/datasets/toy_dataset.html

2. **Cross-validation.** Scikit-learn, evaluating estimator performance.
https://scikit-learn.org/stable/modules/cross_validation.html
3. **Ensembles.** Scikit-learn, gradient boosting and random forests.
<https://scikit-learn.org/stable/modules/ensemble.html>
4. **Support vector classifier settings.** Scikit-learn, SVC reference.
<https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>
5. **Pipelines and column processing.** Scikit-learn, composite estimators.
<https://scikit-learn.org/stable/modules/compose.html>
6. **Clustering and evaluation.** Scikit-learn, clustering user guide.
<https://scikit-learn.org/stable/modules/clustering.html>
7. **Ward linkage and hierarchical merges.** SciPy, linkage reference.
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.cluster.hierarchy.linkage.html>
8. **PCA behaviour and attributes.** Scikit-learn, PCA reference.
<https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>
9. **Anomaly thresholds.** Scikit-learn, Isolation Forest reference.
<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
10. **Self-training.** Scikit-learn, estimator requirements, thresholding, and fitted attributes.
https://scikit-learn.org/stable/modules/generated/sklearn.semi_supervised.SelfTrainingClassifier.html
11. **Label spreading.** Scikit-learn, graph settings, soft clamping, and transduction.
https://scikit-learn.org/stable/modules/generated/sklearn.semi_supervised.LabelSpreading.html
12. **Semi-supervised methods.** Scikit-learn, semi-supervised learning user guide.
https://scikit-learn.org/stable/modules/semi_supervised.html
13. **Mall data source used by the notebook.** CSV hosted in the supplied repository. Its presence here records the exact teaching-data source; the notebook does not establish the dataset's complete collection history.
https://raw.githubusercontent.com/tirthajyoti/Machine-Learning-with-Python/master/Datasets/Mall_Customers.csv